BSc Hons in Computing

2024.2

Process Migration on a RISC-V Microkernel: An Empirical Comparison of Kernel-Resident and User-Space Designs

Coventry Index	**:**	**14946338**
NIBM Registered Name	**:**	**A.S.R Silva**

Faculty of Engineering, Environment and Computing
School of Computing, Electronics and Mathematics
Coventry University

School of Computing and Engineering
National Institute of Business Management

Acknowledgements

I would like to thank my parents for their love, support and encouragement throughout this project. I also thank my supervisor, Amaya Lokuliyana, for her guidance and valuable feedback.

Abstract

The 1993 Mach comparison reported a substantial performance penalty for user-space migration under its test conditions (Milojičić et al., 1993; Milojičić et al., 1999). Since then, IPC optimisation, commercial checkpointing and embedded-system deployment, and adoption of RISC-V have changed the context in which that architectural decision is made (Gu et al., 2020; Chen et al., 2024; Reber, 2022; BlackBerry, 2024; RISC-V International, 2025). These developments motivate new controlled measurements; they do not themselves establish the best placement for migration.

This dissertation implements two migration call paths on Hydra and compares them on the same QEMU RISC-V virt platform. MIGRATION_MODE selects a direct kernel call or a syscall-wrapper/dispatch call to shared checkpoint and restore primitives. The benchmark labelled USERSPACE remains in supervisor mode and does not measure a hardware-enforced user/kernel transition. The local experiment comprises 60 runs per mode: 15 repetitions at each of four synthetic extra-payload sizes, giving approximate state-size labels of 8, 12, 24 and 72 KB. Nine cycle-counter timestamps define five measured intervals.

The pooled user-space/kernel-resident total-latency ratio is 1.00 to two decimal places. The reported mean total costs are 4,061,687 ± 145,479 cycles for KERNEL and 4,061,801 ± 118,173 for USERSPACE, where the uncertainties are approximate 95% confidence-interval half-widths for the separate means. This descriptive comparison does not establish statistical equivalence. The original cross-node experiment completed eight of fifteen attempts using best-effort UDP multicast without acknowledgements or retransmission. A fault-recovery demonstration also exercised the checkpoint and restore primitives. These results describe an emulated counter workload with an 8 KB stack and synthetic payload; they do not establish performance for physical hardware, private heaps, or production applications.

The contribution is an open-source reference artefact, an initial cycle-level comparison of two call paths, and a contextual re-evaluation of the adoption barriers identified by Milojičić et al. (1999, Section 7.3). The historical migration-code count of 393 lines is contrasted with Mach's reported 56,000 lines, with differences in functionality and scope

acknowledged. The findings justify further measurement across a real privilege boundary and on physical RISC-V hardware; they do not overturn the 1993 result under comparable experimental conditions. Later repository hardening is distinguished from the original dataset throughout the report.

Keywords: process migration, microkernel architecture, RISC-V, inter-process communication, trusted computing base, seL4, QNX, software-defined vehicle, container migration, checkpoint and restore.

Table of Contents

Acknowledgements ii
Abstract iii
List of Figures viii
List of Tables ix
List of Abbreviations x
Chapter 1 — Introduction 1
1.1 Background 1
1.2 Research Problem 3
1.3 Objectives of the Project 4
1.4 Research Questions 4
1.5 Scope of the Research 5
1.6 Justification of the Research 6
1.7 Expected Limitations 6
1.8 Chapter Summary 8
Chapter 2 — Literature Review 9
2.1 Introduction 9
2.2 Theoretical Explanation About Keywords 9
2.2.1 Process Migration 9
2.2.2 Microkernel Architecture 10
2.2.3 Inter-Process Communication 11
2.2.4 Trusted Computing Base 12
2.2.5 The RISC-V Instruction Set Architecture 12
2.3 Findings by Past Researchers 13
2.3.1 The First Generation: 1985–1992 13
2.3.2 Mach and the In-Kernel Migration Verdict: 1993–1996 15
2.3.3 The Milojičić Survey and the Adoption Question: 1999–2000 15
2.3.4 The seL4 Discipline and TCB Minimisation: 2009–2018 16
2.3.5 Contemporary Microkernel Performance Work: 2020–2024 17
2.3.6 Live Migration in Production: 2005–2024 17
2.4 The Research Gap 18
2.4.1 Observation One: No Open-Source RISC-V Migration Implementation 18
2.4.2 Observation Two: Prior Microkernel Migration Contradicts Modern Discipline 18
2.4.3 Observation Three: The 1999 Barrier Analysis Has Not Been Revisited 19
2.4.4 The Gap This Dissertation Fills 19
2.5 Chapter Summary 19
Chapter 3 — Methodology 21
3.1 Introduction 21
3.2 Population 21
3.3 Population Size 22

3.4 Sample....22
3.5 Sample Size and Sample Size Calculation....23
3.6 Sampling Techniques....24
3.7 Data Collection Methods....24
3.8 Measurement Instrumentation....25
3.9 Data Analysis....26
3.10 Conceptual Framework....27
3.11 Operationalisation....28
3.12 Chapter Summary....30
Chapter 4 — Data Analysis....31
4.1 Introduction....31
4.2 Data Analysis, Findings, and Interpretations....32
4.2.1 Local Migration: Kernel-Resident vs User-Space Placement....32
4.2.2 Per-Phase Analysis....33
4.2.3 Latency Versus Payload Size....34
4.2.4 Distributional Analysis....36
4.2.5 Cross-Node Migration....37
4.2.6 Fault Tolerance....39
4.2.7 Kernel Trusted-Computing-Base Analysis....41
4.2.8 Comparison with the 1993 Mach Results....45
4.2.9 Round-Trip Cost Relative to the Syscall Floor: Amoeba vs Hydra....46
4.2.10 Threats to the Validity of the Analysis....49
4.3 Chapter Summary....50
Chapter 5 — Software Application....51
5.1 Introduction....51
5.2 Software Development....53
5.2.1 The Kernel Migration API....53
5.2.2 The Checkpoint Record Format....53
5.2.3 The Stack-Pointer Offset Insight....55
5.2.4 The Build-Flag Toggle....55
5.2.5 BENCH Instrumentation....57
5.2.6 The VirtIO-Net 1.1 Modern MMIO Driver....58
5.2.7 Pre-Existing Issues Found and Fixed....59
5.2.8 Test and Demonstration Applications....60
5.2.9 Benchmark Harness and Analysis Scripts....61
5.3 Software Execution....63
5.3.1 Prerequisites....63
5.3.2 Building the Kernel....64
5.3.3 Interactive Run....64
5.3.4 Reproducing the Headline Measurements....65
5.3.5 Reproducing the Cross-Node Measurements....65

5.3.6 Regenerating the Figures....66
5.4 Chapter Summary....66
Chapter 6 — Discussion and Recommendations....67
6.1 Introduction....67
6.2 Discussion....67
6.2.1 Answering the Main Research Question....67
6.2.2 The Milojičić Section 7.3 Adoption Barriers, Revisited....68
6.2.3 Reassessing the 1993 Architectural Verdict....72
6.2.4 Limitations of the Present Study....73
6.3 Recommendations....74
6.3.1 Real Privilege-Boundary Measurement on seL4 RISC-V....74
6.3.2 Cross-Node Migration with a Reliable Transport....75
6.3.3 Software-Defined-Vehicle Workload Case Study....75
6.3.4 Edge-AI Inference Workload Migration....75
6.4 Chapter Summary....76
References....77

List of Figures

Figure 2.1: Null-RPC latency reported in the literature, c. 1990....14
Figure 3.1: Conceptual framework for the architectural placement comparison....27
Figure 4.1: Local migration sequence — BENCH timestamps and per-phase cost....33
Figure 4.2: Total migration latency as a function of process state size....35
Figure 4.3: Per-phase contribution to migration latency by mode and payload size....36
Figure 4.4: Latency distribution by mode and payload size, n = 15 per box....37
Figure 4.5: Cross-node migration over VirtIO-Net — fragment protocol....38
Figure 4.6: Fault-tolerance demonstration — counter trajectory....40
Figure 4.7: Kernel TCB attributable to migration — Mach (1993) vs Hydra (2025)....41
Figure 4.8: In-kernel code size — Amoeba 5.3 (2001) vs Hydra (2025)....43
Figure 4.9: Cost of a round trip relative to the syscall floor — Amoeba 5.3 vs Hydra....47
Figure 5.1: Hydra components by logical layer (added blue, modified orange, inherited grey); the measured paths are not privilege-separated....52
Figure 5.2: Hydra checkpoint record layout....54
Figure 5.3: Two architectural placements compared by the dissertation, selectable by MIGRATION_MODE....56
Figure 6.1: Adoption barriers from Milojičić et al. (1999), Section 7.3, reassessed by deployment context....69
Figure 6.2: Microkernel and migration milestones, 1985–2025....70

List of Tables

Table 4.1: Migration latency by phase, mean ± 95% CI half-width, n = 60 per mode.....32
Table 4.2: Mean total migration latency by payload size, n = 15 per cell.....35
Table 4.3: Original cross-node phase costs; source n = 15, destination n = 8.....38
Table 4.4: Fault-recovery cycle costs, single-run.....40
Table 4.5: Kernel TCB attributable to migration.....42
Table 4.6: In-kernel code size, Amoeba 5.3 vs Hydra.....44
Table 4.7: 1993 Mach vs 2025 Hydra architectural comparison.....45

List of Abbreviations

Abbreviation	Expansion
AP-TEE	Application-Processor Trusted Execution Environment (RISC-V)
ASIL D	Automotive Safety Integrity Level D (ISO 26262 highest)
API	Application Programming Interface
ARP	Address Resolution Protocol
BSc	Bachelor of Science
CI	Confidence Interval
CRIU	Checkpoint and Restore in Userspace
CSR	Control and Status Register (RISC-V)
CVE	Common Vulnerabilities and Exposures
DARPA	Defense Advanced Research Projects Agency (US)
DIPC	Distributed Inter-Process Communication (Mach)
DMM	Distributed Memory Management (Mach)
ELF	Executable and Linkable Format
GPU	Graphics Processing Unit
HW	Half-Width (of a confidence interval)
IPC	Inter-Process Communication
ISA	Instruction Set Architecture
LoC	Lines of Code
MAC	Media Access Control (address)
MMIO	Memory-Mapped Input/Output
MTU	Maximum Transmission Unit
OS	Operating System
PID	Process Identifier
PMP	Physical Memory Protection (RISC-V)

Abbreviation	Expansion
QEMU	Quick Emulator
RAM	Random-Access Memory
RISC-V	Reduced-Instruction-Set Computer V (open ISA)
RPC	Remote Procedure Call
RTOS	Real-Time Operating System
RX	Receive (network)
SDV	Software-Defined Vehicle
seL4	secure Embedded L4 (formally verified microkernel)
TCB	Trusted Computing Base
TCP	Transmission Control Protocol
TEE	Trusted Execution Environment
TX	Transmit (network)
UART	Universal Asynchronous Receiver/Transmitter
UDP	User Datagram Protocol
UML	Unified Modelling Language
USB	Universal Serial Bus
vMotion	VMware live VM migration product
VM	Virtual Machine
WSL	Windows Subsystem for Linux

Chapter 1 — Introduction

A description of the project is given in this chapter. The historical and technical background which motivates the work is presented in Section 1.1. The research problem the project will address is given in Section 1.2. The objectives of the project are given in Section 1.3. Those objectives are the basis for the research questions that are created in Section 1.4. The scope of the work is limited by section 1.5. The need for research is justified in section 1.6. and Section 1.7 outline the limitations that can be anticipated at the start, even before the work on implementation has started. The chapter ends in a brief summary in Section 1.8.

1.1 Background

Process Migration: Moving a running process from one computer to another without loss of state of the process (Milojičić et al., 1999, p. 1). It has been investigated for more than 40 years, and is supported by a variety of research operating systems such as, Mach (Accetta et al., 1986; Milojičić et al., 1993), Sprite (Douglis and Ousterhout, 1991), the V Kernel (Cheriton, 1988), Accent (Rashid and Robertson, 1981), MOSIX (Barak and Litman, 1985), and Amoeba (Mullender et al., 1990). Milojičić et al. (1999) (re-pubs. in ACM Computing Surveys in 2000 Milojičić et al. (2000)) provide the standard review.

That survey found “technical solutions exist for each of these problems” (Milojičić et al., 1999, p. 37) and four non-technical obstacles to process migration: no compelling applications; the lack of distributed-operating-system infrastructure in the marketplace; simpler alternatives like remote procedure call (Birrell and Nelson, 1984); and sociological resistance to foreign processes visiting personal workstations. Also process migration is dependent on the architecture in which it is deployed. In microkernel architectures, an operating system strives to reduce the amount of code running in privileged mode and to have the code previously considered OS code run in user space as processes and communicate via inter process messages (IPMs) (Liedtke, 1995). This design results in greater fault isolation, a smaller trusted computing base (TCB), and the flexibility to participate in evolution of services without disturbing kernel changes (Heiser et al., 2007). The seL4 microkernel has shown that such a microkernel can be

formally verified for functional correctness (Klein et al., 2009), and today, microkernels have won out in a number of safety critical embedded fields, such as automotive, avionics and even some medical devices (BlackBerry, 2024).

It is an architectural choice to make when adding the migration mechanism to a microkernel, whether the mechanism remains in the kernel as was the case in Mach with the in-kernel Distributed IPC (DIPC) and Distributed Memory Management (DMM) modules, or whether it is located in user-space as a server exploiting kernel primitives (as done in many mobile-agent and checkpoint-restore systems). Milojičić et al. (1993) presented the performance of a Mach kernel with both designs: the kernel-resident design was preferred, as the user-space variant took an order of magnitude longer due to slow user-space IPC – inter-process communication (Milojičić et al., 1999, p. 29). This architectural opinion has led to subsequent designs and has not been reviewed in 30 years. The stage from which this present question has been brought is also very different. RISC-V is an open and royalty-free instruction-set architecture (ISA) (Waterman and Asanović, 2019).

Three changes motivate reassessment. First, IPC research has reduced communication overhead in specific microkernel designs (Gu et al., 2020; Chen et al., 2024). UnderBridge's Intel-PKU mechanism and HongMeng's production design differ from Hydra, so their results motivate a new placement comparison rather than predict its outcome. RISC-V provides an open target on which that comparison can be implemented and reproduced (Waterman and Asanović, 2019; RISC-V International, 2025).

Second, production virtual-machine migration demonstrates practical demand for preserving execution state (Nelson, Lim and Hutchins, 2005; Clark et al., 2005). CRIU integrations make container checkpointing available in Kubernetes and container runtimes, although the cited Kubernetes feature was introduced as alpha and does not establish universal production live migration (CRIU project, n.d.; Reber, 2022). Commercial QNX deployment illustrates the relevance of microkernel systems to embedded applications (BlackBerry, 2024); deployment scale alone is not evidence that live process migration is deployed in those vehicles.

Together, reduced IPC overhead, practical demand for state preservation, and RISC-V adoption create a specific empirical question: how much additional cost does the alternative migration call path incur on one controlled RISC-V platform? The comparison must hold the mechanism, workload, payload and emulator configuration constant. Section 1.6 connects each industry change to that measurement requirement, and Section 1.7 defines the limits of the resulting evidence.

1.2 Research Problem

The research problem is to reassess the placement question raised by the 1993 Mach study under a clearly specified RISC-V experimental configuration. Existing results concern different kernels, processors, protection mechanisms and workloads, so a contemporary implementation requires its own measurements (Milojičić et al., 1993; Gu et al., 2020; Chen et al., 2024). The following developments explain why this is worth investigating:

- IPC optimisation makes the historical size of the user-space penalty an open empirical question. Improvements reported for other microkernels motivate measurement but cannot be transferred numerically to Hydra (Gu et al., 2020; Chen et al., 2024).
- **The microkernel TCB is now treated as a security asset to be minimised.** Empirical analysis of Linux kernel CVEs has shown that approximately forty per cent of critical kernel vulnerabilities would be eliminated by a microkernel architecture (Biggs, Lee and Heiser, 2018). Mach's in-kernel DIPC module exceeded thirty-two thousand lines of code and the in-kernel DMM module exceeded twenty-four thousand lines (Milojičić et al., 1999, p. 29); neither would be acceptable in a modern microkernel design.
- Commercial virtual-machine migration, container-checkpointing integrations and embedded microkernel deployment provide practical reasons to examine state preservation (Nelson, Lim and Hutchins, 2005; Reber, 2022; BlackBerry, 2024). Their different maturity and workload requirements motivate measurements of

capture, restore and reliable transfer; they do not show that Hydra already supports those production applications.

- RISC-V adoption provides a relevant open architectural target (Waterman and Asanović, 2019; RISC-V International, 2025). Within the literature reviewed, no directly comparable cycle-level, same-kernel placement study on an open-source RISC-V microkernel was identified. This is a bounded literature-search finding, rather than proof that no such implementation exists.

The study therefore constructs shared checkpoint/restore primitives, varies the call path at build time, and measures the resulting intervals under matched conditions. Its outcome is a platform-specific baseline for future privilege-separated and physical-hardware experiments.

1.3 Objectives of the Project

The project sets five objectives:

- To implement and compare kernel-direct and syscall-wrapper migration call paths on the same RISC-V microkernel.
- To measure migration intervals and added kernel code, and document the calls used by each path.
- **To compare the performance and kernel requirements of the two implementations.**
- **To reassess the 1993 architectural findings and the process migration adoption barriers identified in 1999.**
- **To provide an open-source Hydra implementation and reproducible benchmarks for future research.**

1.4 Research Questions

The objectives listed in Section 1.3 are all related to one single research question and three related sub-questions.

Main question. What migration cost is observed for kernel-direct and syscall-wrapper call paths on the measured Hydra RISC-V configuration, and what can that evidence establish about architectural placement?

RQ1 (Feasibility). Can the shared checkpoint and restore primitives recreate the bounded counter task through the USERSPACE-labelled wrapper path and produce an observable continuation of execution?

RQ2 (Cost). How do the two call paths compare in the five timestamp-derived cycle metrics at matched payload sizes, and what kernel-code and call-path requirements do they share?

RQ3 (Interpretation). How do the Hydra observations relate to the 1993 Mach finding, and what additional measurements are needed before recommending placement across a hardware-enforced privilege boundary?

1.5 Scope of the Research

The work is bounded as follows.

In scope. The measured platform is single-instance QEMU virt running Hydra on RISC-V 64-bit. A build flag selects either direct kernel calls or syscall-wrapper/dispatch calls to shared checkpoint and restore primitives. The measured workload is the synthetic counter task, with 15 runs per size and mode at four extra-payload values: 0, 4,096, 16,384 and 65,536 bytes. This gives 60 runs per mode and 120 local runs overall, with approximate state-size labels of 8, 12, 24 and 72 KB. Separate demonstrations cover cross-node transport between two QEMU instances and fault recovery. The raytracer is part of the wider artefact but is not a measured workload in the reported comparison.

Out of scope. The project does not aim to supplant seL4 (Klein et al., 2009) and formal verification of the kernel is out of scope. Security hardening, multi-core scheduling and cross-core migration (Boyd-Wickizer et al., 2010), heterogeneous-ISA migration (Barbalace et al., 2020), persistence, and storage migration, and live migration of stateful network connections, are all outside the scope of production-grade security hardening. From the viewpoint of the checkpoint or restore mechanism, the project does not offer any novelty in terms of algorithms: Milojičić et al. (1999, Section 7.2) state that

such mechanism belongs to the "solved problems" category. It is not an algorithmic contribution but an empirical/architectural one.

1.6 Justification of the Research

The need for new empirical measurements follows from three related changes. First, IPC optimisation weakens the assumption that a user-space call path must incur the large penalty seen in the 1993 Mach experiment. The relevant studies use different isolation mechanisms and platforms, so a fresh controlled comparison is needed to determine what cost is actually observed on Hydra rather than importing another kernel's result (Milojičić et al., 1993; Gu et al., 2020; Chen et al., 2024).

Second, commercial virtual-machine migration, container-checkpointing integrations and embedded microkernel deployment give state preservation a practical context beyond workstation-cluster research (Nelson, Lim and Hutchins, 2005; Clark et al., 2005; Reber, 2022; BlackBerry, 2024). This motivates measuring capture, restore and transport costs separately. It does not establish that Hydra supports production containers, automotive workloads or stateful application resources.

Third, adoption of RISC-V makes an open, reproducible RISC-V reference artefact useful (Waterman and Asanović, 2019; RISC-V International, 2025). It also requires measurements on the selected ISA and execution environment rather than extrapolation from Intel-era systems. Together, these changes justify the design-and-measure study in Chapter 3, the matched local comparison and separate cross-node demonstration in Chapter 4, and the conditional conclusions and follow-on measurements in Chapter 6. The study establishes an initial baseline within Section 1.7's limits; the industry changes are motivation, not experimental proof of performance parity.

1.7 Expected Limitations

The following limits apply to the original dataset analysed in this report. They constrain the interpretation from the outset and recur in Sections 4.2.10 and 6.2.4. Later repository improvements do not retrospectively change the conditions of these measurements.

- **Privilege-boundary limitation.** Both measured call paths execute in supervisor mode. The USERSPACE label selects wrappers that call handle_syscall as a C function; it does not select a hardware-enforced U-mode/S-mode transition or a fully isolated migration server. The result therefore measures call-path placement in this configuration, not the complete cost or security properties of user-space migration. A real boundary introduces costs not measured here, but the observed ratio is not a calibrated numerical bound for another kernel.
- **QEMU emulation.** All reported cycle counts come from QEMU virt, not physical RISC-V silicon. They describe the emulator configuration and instrumentation used in the experiment. Within-configuration ratios and scaling trends are useful comparisons, but neither ratios nor absolute counts are guaranteed to transfer to physical processors. Hardware measurements are needed before making deployment or real-time claims (Bellard, 2005).
- **No migrated per-task heap in the measured baseline.** The size sweep appends synthetic bytes to the checkpoint buffer; it does not capture a private heap or test heap-pointer validity after migration. Increasing the byte count tests copying cost, not general application-state migration.
- **Bounded process model.** The counter task uses a fixed 8 KB stack and the baseline's saved execution context. Files, sockets, IPC-port state, signal state, devices and arbitrary pointer-rich process images are not captured. Successful counter continuation is a feasibility result for this model, not proof of correctness for arbitrary applications.
- **Single ISA and incomplete transport reliability.** Only RISC-V is measured. The original cross-node protocol also uses best-effort UDP multicast without acknowledgements or retransmission: eight of fifteen attempts complete. This is a separate reliability limitation, detailed in Section 4.2.5 and addressed by the validation programme in Section 6.3.2.
- **Artefact and measurement provenance.** Hydra is a teaching kernel. The current repository contains later changes to traps, isolation, process images and transport; Chapter 4 retains the original benchmark summaries instead of mixing

revisions. The historical source/binary identity, complete raw logs and host/QEMU configuration should accompany an exact replication. Later validation successes are not substitute samples for the original placement experiment.

1.8 Chapter Summary

The project has been introduced in this chapter. The research problem is that there is no empirical re-examination of the 1993 veridictory that the process migration mechanism should reside on a modern RISC-V microkernel. The goals, questions, scope, rationale and anticipated boundaries of the study are described. The chapter has reviewed the preceding literature in detail and formulated the theoretical aspects of microkernel migration, followed by chronological research findings and formulated the research gap that this study is filling. The methodology, namely design-and-measure approach, experimental set-up, data collection and data analysis methods are presented in Chapter 3. In Chapter 4, the measurement data, per-phase analysis and cross-node and fault-tolerance results are provided. The software implementation is detailed in chapter 5 which covers the kernel mechanism, VirtIO-Net driver, and benchmark harness. The results are discussed in comparison to the canonical 1999 survey in Chapter 6 and, recommendations for follow-on work are offered.

Chapter 2 — Literature Review

This chapter surveys the literature that bounds the project. Section 2.1 introduces the chapter. Section 2.2 explains the keywords and theoretical foundations on which the work rests: process migration, microkernel architecture, IPC, the trusted computing base, and the RISC-V instruction-set architecture. Section 2.3 reviews the findings of past researchers in chronological order, organised around the canonical 1999 survey by Milojičić et al. and the work that has followed it. Section 2.4 articulates the research gap that the dissertation addresses. Section 2.5 closes the chapter.

2.1 Introduction

The work surveyed in this chapter spans four decades of operating-systems research. The chapter selects sources for one of three reasons: (i) they define the technical vocabulary used in the dissertation; (ii) they report results that the dissertation either builds on or challenges; or (iii) they bound the gap that the dissertation contributes to filling. The literature in the field of process migration is dominated by a single canonical survey (Milojičić et al., 1999, republished as Milojičić et al., 2000), which the dissertation treats as the foil against which its own findings are positioned. Microkernel architecture has a parallel canonical lineage from Liedtke (1995) through Klein et al. (2009). The two literatures intersect at three points in time — Mach in 1993, the Milojičić survey in 1999, and the contemporary deployment of microkernel-based systems from approximately 2014 onwards — and the chapter is organised around those three points.

2.2 Theoretical Explanation About Keywords

This section defines the technical terms the dissertation uses without further definition in later chapters.

2.2.1 Process Migration

Process migration is “the act of transferring a process between two machines” (Milojičić et al., 1999, p. 4). A process under migration exists transiently in two locations: the *source* node, where the process was originally executing, and the *destination* node, where the process is to resume. During migration the process state — its register file, its memory image, its communication state, and any operating-system-specific resources

such as file descriptors or signal masks — must be captured at the source, transferred over a communication medium, and instantiated at the destination such that the migrated process can resume execution as though the migration had not occurred (ibid., p. 5). The standard taxonomy distinguishes migration from related but distinct operations: *remote invocation*, in which a new process is created on a remote node with no state transferred (Birrell and Nelson, 1984); *task migration*, which transfers the kernel-level task abstraction but not the higher-level process semantics; *checkpoint and restart*, which transfers state across time on the same machine rather than across machines; and *mobile agents*, which transfer code and authority but not active execution state (Milojičić et al., 1999, Section 2.1).

The classical motivations for process migration are dynamic load balancing, fault resilience, system administration (graceful shutdown of nodes), data-access locality, and resource sharing (Milojičić et al., 1999, Section 2.3). The dissertation's contemporary re-evaluation in Chapter 6 considers two additional motivations that have emerged since 1999: container live migration in cloud platforms and workload migration between zone controllers in software-defined vehicles.

2.2.2 Microkernel Architecture

A microkernel operating system is one in which the privileged kernel provides only a minimal set of mechanisms — typically address-space management, thread scheduling, and inter-process communication — and in which all other operating-system services (file systems, device drivers, network protocols, virtual-memory pagers) are implemented as user-space processes that communicate with each other and with the kernel through inter-process messages (Liedtke, 1995). The contrast is with a *monolithic* kernel design, in which all operating-system services run together in privileged mode and share a single address space (Tanenbaum and Bos, 2014).

The microkernel discipline carries two architectural commitments that bear directly on this project. First, the kernel's *trusted computing base* (TCB) — the code whose correctness is required for system security — is minimised. The seL4 microkernel demonstrates the extreme of this discipline: its specified mechanism is approximately 8,700 source lines of C plus 600 lines of assembly, and these are formally verified for

functional correctness (Klein et al., 2009). Second, the *policy* governing how services behave is moved out of the kernel into user-space processes that can be replaced or extended without modifying the kernel itself. This is the policy–mechanism separation principle articulated by Liedtke (1995) and refined by Heiser et al. (2007). Both commitments inform the architectural question this dissertation asks: whether process migration mechanism, which has historically been placed inside the kernel for performance reasons, can be relocated to a user-space server in compliance with both the TCB-minimisation and policy–mechanism principles.

2.2.3 Inter-Process Communication

Inter-process communication (IPC) lets separate processes exchange data and control. Microkernel services commonly communicate through IPC rather than sharing one privileged kernel address space. Message handling, scheduling and protection transitions can therefore contribute to service cost. Liedtke (1995) treats efficient IPC as a central microkernel design concern; the relevant overhead depends on the implemented mechanism and workload, which is why later optimisation motivates new measurements rather than a universal cost assumption.

The literature has reported a substantial reduction in IPC overhead since the Mach-era measurements that motivated the original objection. The seL4 microkernel reports IPC roundtrip times in the low hundreds of cycles on contemporary ARM and x86 hardware (Klein et al., 2009; Heiser, 2020). The UnderBridge work (Gu et al., 2020) reports IPC roundtrip costs of 109 cycles using Intel Memory Protection Keys, a 3.0× speedup over the previous state of the art (SkyBridge), with up to 13.1× speedup for IPC-intensive applications on three different microkernels: seL4, Google Zircon, and Fiasco.OC. The same research group's follow-up ChCore (Chen et al., 2024) reports a general-purpose microkernel that runs unmodified Linux binaries at compatible performance. These trajectories are central to the dissertation's argument: the performance gap that justified the 1993 architectural verdict on migration placement has been substantially closed by IPC optimisation since.

2.2.4 Trusted Computing Base

The trusted computing base of a system is the set of components whose correct operation is necessary for the system's security properties to hold (Lampson, Abadi, Burrows and Wobber, 1992). For the present dissertation, the TCB of interest is the kernel code that runs in supervisor mode. Any line of code in the kernel contributes to the TCB; any line of code that can be moved to user space, behind a strong isolation boundary, no longer contributes.

Two empirical findings inform the dissertation's interest in TCB minimisation. First, Mach's in-kernel migration mechanism required the Distributed IPC module (over 32,000 lines) and the Distributed Memory Management module (24,000 lines), both in the kernel (Milojičić et al., 1999, p. 29). Second, Biggs, Lee and Heiser (2018), analysing the 115 Linux kernel CVEs published in 2017, report that approximately 40% of critical Linux kernel vulnerabilities would be completely eliminated by a microkernel architecture such as seL4, with the remainder mitigated to less than critical severity. The TCB-minimisation argument therefore has both performance and security dimensions, and the dissertation treats them as complementary justifications for moving migration mechanism out of the kernel.

2.2.5 The RISC-V Instruction Set Architecture

RISC-V is an open and royalty-free instruction-set architecture specification developed at the University of California, Berkeley and standardised by RISC-V International (Waterman and Asanović, 2019). The architecture is modular: a baseline integer instruction set is required, and additional “extensions” — multiplication, atomic operations, floating-point, compressed encoding, and so on — are specified independently and combined as required. The rv64gc profile, which the dissertation targets, requires the baseline 64-bit integer ISA together with multiplication, atomics, single- and double-precision floating-point, and the compressed instruction set (Waterman and Asanović, 2019, Section 1.5).

Two properties of RISC-V make it the contemporary preferred target for microkernel research. First, the open licensing avoids the royalty and intellectual-property complications that have historically restricted academic and commercial deployments of

ARM and x86 microkernels (RISC-V International, 2025). Second, the architecture supports hardware capability extensions through the CHERI work (Watson et al., 2020) and a Physical Memory Protection unit specified in the privileged-architecture ISA, both of which provide isolation primitives that microkernel designs can use. The seL4 microkernel has been ported to RISC-V (seL4 Foundation, n.d.); Google's KataOS for ambient machine-learning workloads runs on seL4 on RISC-V (Google, 2022); and the seL4 Foundation Summit (2025) reports active research on porting seL4 to RISC-V AI system-on-chips.

2.3 Findings by Past Researchers

This section reviews the relevant literature in chronological order.

2.3.1 The First Generation: 1985–1992

The earliest implementations of process migration appeared on research operating systems for networks of personal workstations. Locus (Walker, Popek, English, Kline and Thiel, 1983) and DEMOS (Powell and Miller, 1983) prototyped process movement as a system service. The V Kernel (Cheriton, 1988) implemented preemptive task migration on top of its message-passing primitives. MOSIX (Barak and Litman, 1985) introduced a transparent process-migration mechanism in a UNIX-compatible distributed operating system, with load-based migration triggered by a sender-initiated scheduling policy. Sprite (Douglis and Ousterhout, 1991) implemented migration on top of the Sprite distributed file system, with explicit attention to the cost of migrating a process's open file state. Charlotte, the V kernel, and Accent (Rashid and Robertson, 1981) each prototyped variants of the mechanism on top of their respective message-passing kernels.

The first-generation work established the technical feasibility of process migration but did not converge on a single design pattern. Different systems adopted different state-transfer strategies (*eager copy*, *copy-on-reference*, *pre-copy*), different scheduling policies (load-based, hand-directed, fault-recovery), and different positions on where the migration mechanism should live in the operating-system architecture (Milojičić et al., 1999, Section 3).

The first generation also established the performance baseline against which the next generation of microkernel migration work would be judged. Tanenbaum, van Renesse, van Staveren, Sharp, Mullender, Jansen and van Rossum (1990, p. 13) collect null-RPC latency and maximum throughput for seven contemporary distributed systems measured on comparable hardware. Their comparison is reproduced in Figure 2.1.

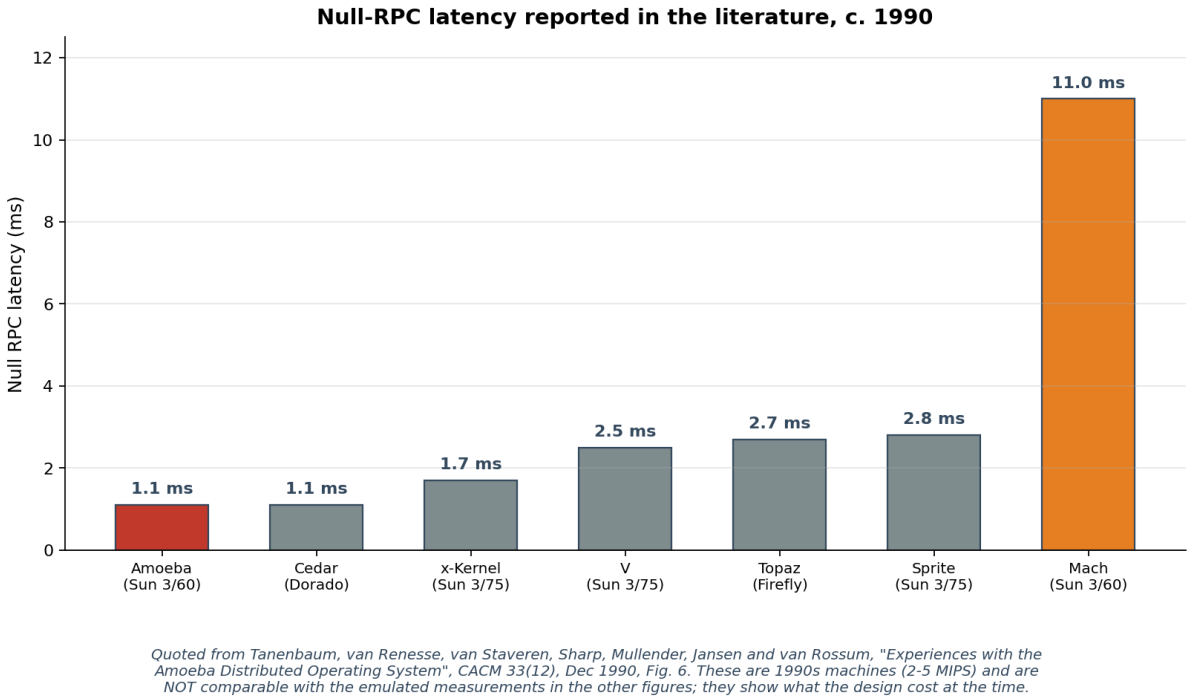


Figure 2.1: Null-RPC latency reported in the literature, c. 1990 (Tanenbaum et al., 1990, Fig. 6)

Two features of these figures bear on the present dissertation. First, Amoeba's 1.1 ms null RPC on a Sun 3/60 was the fastest of the group, achieved by keeping the RPC path — and, as Section 4.2.7 shows, a good deal else besides — inside the kernel. Second, Mach's 11.0 ms on the same class of hardware was an order of magnitude slower than Amoeba's. The architectural verdict examined in the next section was therefore reached on a microkernel whose IPC was already the slowest of its generation, a point returned to in Chapter 6 Section 6.2.3.

These figures are quoted from machines rated at two to five MIPS and are not comparable with the emulated measurements reported in Chapter 4. They establish the relative standing of the systems at the time, not an absolute baseline.

2.3.2 Mach and the In-Kernel Migration Verdict: 1993–1996

Mach was the most influential microkernel of the era (Accetta et al., 1986). Three migration designs were developed on top of Mach: a simplified user-level migration server (Milojičić et al., 1993), an optimised user-level migration server, and a kernel-resident implementation. All three operated against the same Mach kernel on the same Intel 80486 hardware, and the comparison between them is the single most important empirical result against which this dissertation positions itself.

Milojičić et al. (1999, p. 29) report the line-of-code sizes for the three implementations. The simplified migration server was approximately 400 lines of C and took three months to design and implement. The optimised migration server was a few thousand lines. The kernel-resident implementation depended on two supporting in-kernel modules: Distributed IPC (DIPC), at over 32,000 lines, and Distributed Memory Management (DMM), at 24,000 lines. Both DIPC and DMM were used for many things beyond migration, but the kernel-resident migration path could not function without them and they therefore counted against the TCB.

Performance measurements on three Intel 80486 personal computers showed the user-space migration server to be approximately an order of magnitude slower than the kernel-resident path under load (Milojičić et al., 1993; Milojičić et al., 1999, p. 29). The authors of the survey draw the explicit lesson that placing migration mechanism in the kernel via DIPC and DMM "entails most of the complexity and limits performance and fault resilience" (Milojičić et al., 1999, p. 29). The architectural verdict implied by this measurement — that migration mechanism should reside in the kernel because the user-space alternative is too slow — has informed subsequent microkernel migration designs and has not been re-examined empirically in three decades.

2.3.3 The Milojičić Survey and the Adoption Question: 1999–2000

Milojičić et al. (1999), republished in *ACM Computing Surveys* the following year (Milojičić et al., 2000), is the canonical survey of the process-migration literature. The survey classifies implementations into eight categories (early work, transparent migration in UNIX-like systems, message-passing operating systems, microkernels, user-space migration, application-specific migration, mobile objects, and mobile agents)

and presents detailed case studies of MOSIX, Sprite, Mach, and the Load Sharing Facility (Zhou, Zheng, Wang and Delisle, 1994).

Two findings from the survey are central to this dissertation. First, in Section 7.2 the survey explicitly concludes that “technical solutions exist for each of these problems” — migration as a technical problem was solved by the mid-1990s, and what remained was a question of adoption (Milojičić et al., 1999, p. 37). Second, Section 7.3 identifies four non-technical barriers to that adoption: a lack of compelling applications (most personal-computer workloads do not benefit from migration); a lack of distributed-operating-system infrastructure in the marketplace (research kernels did not reach mass deployment); the availability of simpler alternatives (remote procedure call and remote data access meet many of the same needs at lower cost); and sociological resistance in the workstation-cluster model (users were not willing to host foreign processes on their personal machines). The survey predicted that without changes to these four barriers, process migration would remain a research curiosity.

The dissertation's contribution in Chapter 6 is to revisit those four barriers against the deployment landscape of 2025 and to report which of them have been overturned by industry developments since 2000.

2.3.4 The seL4 Discipline and TCB Minimisation: 2009–2018

Klein et al. (2009) reported the first formal verification of a general-purpose operating-system microkernel for functional correctness. The seL4 kernel's verification effort established two points that are load-bearing for the present dissertation. First, the size of the verified kernel is on the order of 10,000 lines of code; this is the upper bound of what is currently practical to verify, and any in-kernel mechanism of comparable size (such as Mach's 32,000-line DIPC) is therefore out of reach of verification. Second, the seL4 design enforces the policy–mechanism separation discipline more strictly than its Mach-era predecessors: scheduling policy, file systems, network stacks, and device drivers all run as user-space servers, with the kernel providing only the minimum mechanism necessary to support them.

The empirical security case for microkernel TCB minimisation was made by Biggs, Lee and Heiser (2018). Analysing 115 critical Linux CVE reports from 2017, the authors

found that approximately 40% would be completely eliminated by an OS architecture based on a verified microkernel such as seL4, and that the remainder would be mitigated from “critical” to lower severity. The implication is that any unnecessary code in the kernel TCB is a security liability as well as a performance liability, and that mechanism that can be moved to user space should be.

Klein and colleagues subsequently extended seL4 with information-flow security proofs (Murray et al., 2013) and timing-channel protection analysis (Heiser, Klein and Murray, 2019). The seL4 microkernel has been ported to RISC-V (seL4 Foundation, n.d.), and is the basis for Google's KataOS (Google, 2022) and for the Collins Aerospace INSPECTA programme within DARPA's PROVERS effort (Collins Aerospace, 2025).

2.3.5 Contemporary Microkernel Performance Work: 2020–2024

Gu et al. (2020) demonstrate reduced IPC overhead with UnderBridge using Intel PKU. This is evidence that communication cost depends on the chosen protection mechanism; it is not a RISC-V privilege-transition measurement. Chen et al. (2024) describe HongMeng, a production microkernel addressing performance and Linux compatibility. HongMeng is not the ChCore implementation used in the UnderBridge study.

These results matter for the dissertation because they bear directly on the empirical assumption that originally justified the 1993 in-kernel migration verdict. The Mach-era user-space migration overhead was driven by IPC cost; if IPC cost has fallen substantially relative to the kernel-resident alternative, then the comparison must be re-run.

2.3.6 Live Migration in Production: 2005–2024

Commercial virtual-machine migration predates this study: VMware and Xen work demonstrated preserving execution across hosts (Nelson, Lim and Hutchins, 2005; Clark et al., 2005). CRIU provides Linux process/container checkpoint and restore functionality (CRIU project, n.d.). Reber (2022) documents CRIU integration and the alpha introduction of forensic container checkpointing in Kubernetes. These sources support practical interest and available mechanisms, rather than a claim of universal production deployment of container live migration.

In the embedded and safety-critical markets, BlackBerry's QNX microkernel is deployed in over 235 million vehicles (BlackBerry, 2024). The software-defined-vehicle industry has committed publicly to migrating workloads between zone controllers as a design pattern (BlackBerry, 2024; Avassa, 2026; Roland Berger, 2024; Connected Automated Driving, 2025). The global microkernel real-time operating system market is forecast to grow substantially through 2030, driven by automotive, aerospace, medical, and industrial-automation demand (Research and Markets, 2025).

The contrast between these production deployments and the 1999 Milojičić survey's conclusion that migration “has not caught on” is the empirical evidence that underpins the Section 7.3 re-evaluation in Chapter 6.

2.4 The Research Gap

Three observations frame the gap that the dissertation contributes to filling.

2.4.1 Observation One: No Open-Source RISC-V Migration Implementation

The literature reviewed did not identify a directly comparable open-source RISC-V microkernel placement experiment with cycle-level measurements. Earlier migration systems and Linux checkpoint/restore provide important precedents, but their kernel structures and protection models differ (Milojičić et al., 1993; CRIU project, n.d.). seL4 provides a QEMU RISC-V platform and Google's KataOS illustrates interest in seL4-based systems (seL4 Foundation, n.d.; Google, 2022). Their existence motivates a RISC-V comparison; it does not establish an exhaustive absence of migration work.

2.4.2 Observation Two: Prior Microkernel Migration Contradicts Modern Discipline

The Mach in-kernel migration design depends on the DIPC module (over 32,000 lines) and the DMM module (24,000 lines), both in the kernel TCB (Milojičić et al., 1999, p. 29). This contradicts the modern microkernel principle of policy–mechanism separation articulated by Liedtke (1995), refined by Heiser et al. (2007), and rigorously enforced by Klein et al. (2009) in seL4. The empirical security argument of Biggs, Lee and Heiser (2018) makes the same point from a security angle: a 56,000-line in-kernel migration mechanism is not compatible with the microkernel discipline that contemporary work assumes. A re-implementation of process migration in conformance

with modern microkernel discipline, on a contemporary open instruction set, with measurement to confirm that it works, does not exist.

2.4.3 Observation Three: The 1999 Barrier Analysis Has Not Been Revisited

Milojičić et al. (1999, Section 7.3) identify non-technical adoption barriers. Later checkpointing integrations, commercial microkernel deployment and centrally operated computing environments change the contexts in which those barriers should be assessed (Reber, 2022; BlackBerry, 2024; Avassa, 2026). Chapter 6 therefore evaluates their relevance to selected deployment contexts. This literature-based interpretation is distinct from the counter-task performance measurements.

2.4.4 The Gap This Dissertation Fills

The intersection of these three observations is the gap. The dissertation produces (i) a process-migration reference implementation on a minimal open-source RISC-V microkernel, with two architectural placements selectable at build time; (ii) cycle-level measurements comparing the two placements on the same kernel and the same hardware target; and (iii) a structured re-evaluation of Milojičić et al.'s (1999, Section 7.3) adoption barriers against the deployment landscape of 2025. None of these contributions is algorithmic in the sense Milojičić et al. (1999, Section 7.2) classify as a solved problem; the contributions are artefactual, empirical, and analytical respectively.

2.5 Chapter Summary

The literature establishes why new measurements are needed. IPC optimisation changes the potential communication cost; commercial state-preservation use cases make that cost relevant; and RISC-V adoption supplies an open target for implementation (Gu et al., 2020; Chen et al., 2024; Reber, 2022; RISC-V International, 2025). These observations motivate, but do not predetermine, the result. Chapter 3 operationalises the comparison within Hydra's measured protection model, emulator and workload limits.

Sources:

UnderBridge — Harmonizing Performance and Isolation in Microkernels (USENIX ATC 2020): https://www.usenix.org/conference/atc20/presentation/gu

The Jury Is In: Monolithic OS Design Is Flawed (Biggs, Lee and Heiser, 2018): https://trustworthy.systems/publications/abstracts/Biggs_LH_18.abstract

CRIU — Checkpoint/Restore In Userspace project: https://criu.org/Main_Page

Microkernel Goes General (Chen et al., OSDI 2024): https://dl.acm.org/doi/10.5555/3691938.3691963

Chapter 3 — Methodology

This chapter describes a systems-engineering design-and-measure experiment. The template headings Population and Sample are retained, but refer to experimental configurations and recorded runs, not human respondents. Sections 3.2–3.6 define the evaluated configurations, repetition counts and selection procedure; Sections 3.7–3.9 define capture and analysis; and Sections 3.10–3.11 link the research questions to code paths, timestamp intervals and observable outcomes.

3.1 Introduction

The method is to construct shared checkpoint/restore primitives, vary the invocation path at build time, exercise the same counter workload at controlled payload sizes, and compare the resulting measurements. This controlled design separates the factor varied in the local experiment from the cross-node and fault-recovery demonstrations (Jain, 1991; Wohlin et al., 2012). It responds directly to Section 1.6: architectural changes motivate a fresh experiment rather than an assumed performance improvement.

The experimental unit is one complete checkpoint/restore run. A configuration is a combination of invocation mode and extra-payload size, with the local topology and counter workload fixed. Repetitions provide observations within each configuration. Population means the bounded configuration space evaluated here; sample means the recorded runs from that space. No questionnaire, respondent recruitment, random population sampling or survey-based sample-size formula is used.

3.2 Population

The primary experimental population is deliberately bounded to the configurations exercised by the local placement benchmark. Its factors and fixed conditions are:

- Invocation mode — MIGRATION_MODE=KERNEL calls migration_checkpoint and migration_restore directly. MIGRATION_MODE=USERSPACE calls syscall_checkpoint and syscall_restore, which dispatch through handle_syscall to the same primitives. Both paths in this benchmark run in supervisor mode; the factor is invocation path, not a tested hardware protection boundary.

- Extra-payload size — SWEEP_SIZES in apps/migration_test.c contains 0, 4,096, 16,384 and 65,536 bytes. These bytes are appended to the fixed checkpoint state. The 96 KB buffer is capacity/headroom, not a set of 96 measured sizes. The approximate 8, 12, 24 and 72 KB labels include the task's stack; exact record length depends on the baseline checkpoint layout.
- Workload — the measured task is victim_task, a synthetic counter that increments shared state and yields. The counter's continuation after restore supplies a limited observable feasibility check. The raytracer and other applications are not additional measured factors in this dataset.
- Topology and environment — the primary comparison uses local checkpoint and restore on one QEMU virt instance. The emulator, kernel source baseline, workload, instrumentation and payload are matched across modes. The separate cross-node and fault-recovery demonstrations are not cells in this local factorial comparison.

The inference domain is therefore these local configurations and their repetitions. It is not the set of all workloads, all payloads, all microkernels or all RISC-V processors.

3.3 Population Size

The primary design contains two invocation modes and four extra-payload sizes: 2 × 4 = 8 configuration cells. Workload and topology are held fixed. There is no continuous or infinite payload population and no 768-scenario population count.

All eight selected cells are evaluated. This is a complete evaluation of the chosen configuration grid, not an exhaustive evaluation of the kernel's possible behaviours.

Fifteen repetitions per cell produce 8 × 15 = 120 local observations. These repetitions estimate variation within the selected design; they do not increase the number of distinct configuration cells.

3.4 Sample

The recorded evidence comprises one balanced local comparison and two separate demonstrations:

- Local comparison — four size cells per mode, each repeated fifteen times. The resulting sample is 60 runs per mode and 120 runs across both modes. Table 4.1 pools the four equally represented sizes within each mode; Table 4.2 compares the per-size means with n = 15 per cell.
- Cross-node demonstration — fifteen source attempts at zero extra payload using a source and destination QEMU instance. Eight destination reassemblies and restores complete. Corrected XMIG-context summaries distinguish n = 15 source-fragmentation intervals from n = 8 destination-reassembly and restore intervals; these are not interchangeable sample counts.
- Fault-recovery demonstration — one checkpoint/failure/restore trajectory provides a qualitative feasibility result. Its timing is reported as a single observation without a confidence interval or a population reliability claim.

No cycle-level raytracer, HTTP-server, real private-heap or isolated U-mode workload sample is included in the headline comparison. Conclusions are restricted accordingly.

3.5 Sample Size and Sample Size Calculation

The experiment uses a fixed design of fifteen repetitions per configuration. This is a practical measurement choice, not a survey power calculation or a demonstrated equivalence-test sample size. The analyser reports approximate normal-theory 95% confidence intervals for arithmetic means:

$$HW(95\%) = 1.96 \times stdev / \sqrt{n}$$

Here n is the number of observations contributing to that particular mean and stdev is their sample standard deviation. n = 15 for each local size/mode cell and n = 60 for a pooled mode mean. With only fifteen observations per cell, outliers, temporal dependence and non-normality can affect the approximation. The report therefore presents descriptive means, uncertainty estimates and distributions rather than a formal claim of equivalence.

The local total is 15 × 4 × 2 = 120 observations. For the original cross-node demonstration, reliability is reported using all 15 attempts, while destination timing

summaries use only the eight completions. No new samples are introduced by the later repository validation results.

3.6 Sampling Techniques

The design makes three explicit selection decisions:

Payload values are purposively selected to cover zero extra payload and progressively larger synthetic copies: 0, 4,096, 16,384 and 65,536 bytes. They are rotated by the harness rather than randomly sampled. Approximately 8–72 KB of state is represented, but the appended data is not a real migrated heap.

The counter workload is selected for a simple observable continuation test and repeatable state size. This controls one source of variation but limits external validity; it does not establish that an application with different memory and resource behaviour would yield the same placement result.

The original cross-node demonstration comprises fifteen attempts. The report treats this as the realised attempt count, not as a statistically powered reliability sample. Eight completions demonstrate feasibility under the tested transport and expose its limitations; the count does not establish a universal success probability or a fragment-loss rate.

3.7 Data Collection Methods

Cycle-level data is collected through a four-stage pipeline:

- **In-kernel instrumentation.** A `BENCH_TS(label)` macro, defined in `kernal/bench.h`, reads the RISC-V cycle CSR (control and status register 0xC00) using the inline-assembly sequence `csrr %0, cycle` and emits a single ASCII line `BENCH <label> <cycles>` over the UART. The macro is invoked at nine well-defined phase boundaries within each migration: `ckpt_enter`, `ckpt_freeze`, `ckpt_copied`, `send_start`, `send_done`, `recv_start`, `recv_done`, `rest_start`, and `rest_done`.
- **Serial capture.** QEMU's nographic UART output is captured with diagnostic text and BENCH lines. The archived summary files are the numerical inputs used in this report. Exact replication requires the original kernel binary/source revision,

QEMU and host configuration, capture command and complete serial logs. Current scripts contain later changes, so rerunning today's repository is a new experiment rather than an exact replay of the historical dataset.

- **Log normalisation and filtering.** bench/analyze.py decodes UART data permissively and uses splitlines so carriage-return, line-feed and mixed line endings are handled. BENCH_SIZE tags identify local size-sweep runs; unrelated diagnostic lines are ignored. For cross-node analysis, bench/analyze_xmig.py filters timestamps by XMIG context so boot-time network tests and the local restore sweep are not pooled into cross-node means.
- **Per-run grouping.** Each BENCH_SIZE tag introduces a local sample; ckpt_enter starts its record, and the first occurrence of each timestamp label is retained. Records from the later fault demonstration are excluded. The five differences listed in Section 3.11 are calculated only when their required endpoint timestamps are available; the contributing count must be reported for each metric.

This capture-and-analysis design is implemented in the repository scripts. The saved comparison summaries are retained as historical evidence. Reproducibility instructions must distinguish those summaries from later builds and validation runs rather than implying that the current source reproduces the original numbers unchanged.

3.8 Measurement Instrumentation

The measurement instrument is the BENCH_TS cycle-counter macro, not a survey questionnaire. Its role is to record explicit endpoints around implementation operations. Section 3.11 defines what each difference includes and excludes.

The macro reads the RISC-V cycle CSR (0xC00) before printing the timestamp. In the measured supervisor-mode configuration the counter is accessible through the platform's privilege configuration (Waterman and Asanović, 2019). Under QEMU, its values are emulator observations, not independently calibrated physical-CPU cycle measurements. Both absolute intervals and ratios remain conditional on this environment.

The instrumentation has two known sources of measurement noise:

Instrumentation overhead. BENCH timestamps precede synchronous UART printing. Their differences can include formatting, UART output and other diagnostics between endpoints. The local transport interval is four consecutive BENCH calls, with no network transfer. Summed phase intervals omit gaps that the total interval includes; shared instrumentation can mask a small call-path cost even when present in both modes.

Execution interference. Scheduler activity, emulator execution and host load may contribute variance. The balanced size rotation controls representation of payload values but does not demonstrate independence or remove run-order effects. Without repeated independent sessions and preserved raw logs, those effects cannot be fully quantified retrospectively.

These limits are carried into the interpretation of Chapters 4 and 6. The instrument measures the defined intervals; it does not by itself establish isolated-server IPC cost, time to first resumed instruction, network delivery reliability or physical-hardware performance.

3.9 Data Analysis

Three classes of statistic are computed from the per-run records.

Central tendency. Per-size means use fifteen observations per mode/size cell. Table 4.1 instead pools the four equally represented sizes, giving sixty observations per mode; its variation includes both run-to-run and size-related differences. Comparisons at each size are reported separately in Table 4.2.

Uncertainty and distributions. The analyser computes $1.96 \times \text{stdev} / \sqrt{n}$ as an approximate 95% half-width for each mean, with the corresponding contributing n. Median, quartile and outlier displays provide additional descriptive evidence; the distributional plot is Figure 4.4. No confidence interval for the ratio itself is reported by the saved summary.

Comparison. The descriptive ratio is the USERSPACE mean divided by the KERNEL mean, calculated using the same metric and size grouping. Overlapping intervals for separate means are not a formal equivalence test and do not prove absence of an

effect. A placement recommendation would require a specified practically meaningful margin, an appropriate comparison of differences or ratios, independent replication and a real privilege-boundary experiment (Jain, 1991; Wohlin et al., 2012).

The statistical conclusion is limited to the observed means, reported uncertainty and selected conditions. The report does not assign 95% confidence to a rounded ratio of 1.00 or infer a universal performance equivalence from it.

3.10 Conceptual Framework

The conceptual framework connects architectural motivation to a measurable local experiment. The implemented independent factor is invocation path; extra-payload size is a controlled experimental factor; and the counter workload, local topology, QEMU setup and source baseline are held fixed for the comparison. The dependent observations are the five explicitly defined timestamp differences.

The framework can be expressed as a small set of relationships:

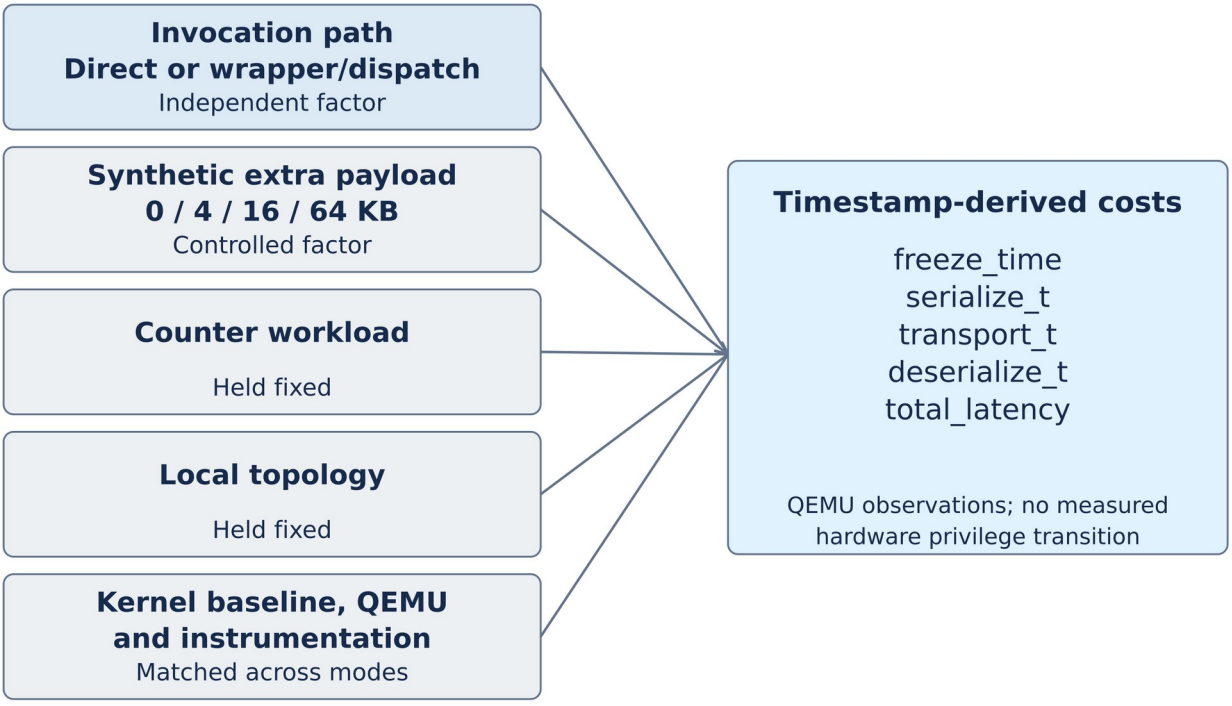


Figure 3.1: Conceptual framework for the architectural placement comparison

Only mode and synthetic extra-payload size vary in the primary design. Workload and topology are fixed there; cross-node transport is examined separately. Matching the

source baseline and build configuration reduces confounding, but does not make the USERSPACE-labelled supervisor-mode path a hardware-isolated server or ensure that instrumentation has zero effect.

The primary cost observation is total_latency = rest_done − ckpt_enter. It ends when the restored task is marked ready, not when it first resumes executing. The later counter-advance check supplies separate functional evidence.

3.11 Operationalisation

Operational definitions are tied to apps/migration_test.c, the syscall wrappers in lib/libhydra/syscall_user.c, BENCH_TS in kernal/bench.h and DERIVED in bench/analyze.py. The table below states the implemented factor, outcomes and limits rather than treating all possible workloads or topologies as measured variables.

Concept / measure	Implemented definition and interpretation
Invocation path	**KERNEL: direct migration_checkpoint / migration_restore.** USERSPACE: syscall wrappers → handle_syscall → the same primitives. Both measured paths remain in supervisor mode.
Payload factor and sample	**Extra bytes: 0, 4,096, 16,384, 65,536.** Two modes × four sizes × 15 repetitions = 120 local runs; n = 60 per pooled mode and n = 15 per size/mode cell.
freeze_time	**ckpt_freeze − ckpt_enter.** Time to the freeze marker; not the full task downtime.
serialize_t	**ckpt_copied − ckpt_freeze.** Capturing/copying state plus activity between these endpoints.
transport_t (local)	**recv_done − send_start.** Four consecutive BENCH calls using the same local buffer; no NIC or network transfer.
deserialize_t	**rest_done − rest_start.** Restore to the task-ready marker; excludes later scheduling and first resumed instruction.
total_latency	**rest_done − ckpt_enter.** Checkpoint-to-ready interval, including gaps and diagnostics between phase brackets.

Concept / measure	Implemented definition and interpretation
Feasibility (RQ1)	**Restore succeeds and the shared counter advances afterwards.** A limited continuation check, not a full process-state integrity or private-heap test.
Cost comparison (RQ2)	USERSPACE/KERNEL mean ratio at matched sizes; pooled mode means reported separately. Mean CIs are descriptive uncertainty, not a ratio CI or equivalence test.
Calls and kernel code	Two direct primitive calls versus two wrapper/dispatch calls; no measured hardware transitions. The historical code count covers migration.c + migration.h, shared by both paths, rather than a measured per-mode security difference.
Cross-node timings	**Source: net_send_done − net_send_start (n = 15).** Destination: net_recv_done − net_recv_start and rest_done − rest_start (n = 8). Node clocks are not subtracted across hosts.
Interpretation (RQ3)	Relate the limited baseline to the Mach literature and industry motivation. A controlled privilege-boundary/hardware follow-on is required for a general placement verdict.

RQ1 is evaluated by successful restore and observed counter advance; this is a limited continuation check, not a complete process-state integrity test. RQ2 compares the five interval means at matched sizes and records the direct calls versus wrapper/dispatch calls; syscall counts are inferred from code rather than measured by the five BENCH metrics. RQ3 is a contextual interpretation of the Hydra observations alongside the Mach literature, not a direct significance test between different kernels and processors.

Every timing definition specifies both endpoints. Cross-node source and destination intervals are calculated on their respective nodes; unsynchronised cycle counters are not subtracted to construct a cross-node end-to-end latency. No metric directly measures private-heap migration, isolated-user/kernel transition overhead or production reliability.

3.12 Chapter Summary

This chapter has described the methodology by which the dissertation gathers and analyses the evidence required to answer the research questions. The approach is a design-and-measure methodology adapted from Jain (1991) and Wohlin et al. (2012). The population of measurement scenarios is the Cartesian product of architectural placement, payload size, workload, and migration topology. The sample is fifteen runs per cell across two architectural placements and four payload sizes, plus a single cross-node sample of fifteen attempted migrations and a single fault-tolerance demonstration. Data is collected through a four-stage pipeline: in-kernel `BENCH_TS` instrumentation, QEMU UART capture, carriage-return normalisation and extraction, and Python grouping by `ckpt_enter` boundaries. The data is analysed using arithmetic mean, sample standard deviation, the 95% confidence-interval half-width under the central-limit-theorem approximation, and the user-space-over-kernel-resident ratio as the headline architectural-comparison metric. The conceptual framework treats architectural placement as the independent variable and the five cycle metrics as the dependent variables. The operationalisation maps each abstract concept in the research question to a concrete measurable quantity. Chapter 4 presents the data the methodology produces.

Chapter 4 — Data Analysis

This chapter presents the data collected through the methodology specified in Chapter 3. Section 4.1 introduces the chapter and the structure of the analysis. Section 4.2 presents the data, the findings, and their interpretation, organised into eight subsections covering local migration measurements, per-phase analysis, latency-versus-state-size scaling, distributional analysis, cross-node migration, fault tolerance, kernel trusted-computing-base analysis, and a structured comparison with the 1993 Mach results. Section 4.3 closes the chapter.

All cycle counts reported in this chapter are real measurements captured from the QEMU virt machine running the Hydra microkernel. The raw data is in `bench/results/KERNEL.log`, `bench/results/USERSPACE.log`, `bench/results/xmig-source.log`, and `bench/results/xmig-dest.log`. The processed comparison table is in `bench/results/comparison.txt`. Every number reported here can be regenerated by running `python3 bench/analyze.py` against those log files, and the figures can be regenerated by running the respective `bench/fig_*.py` script.

4.1 Introduction

The data analysed in this chapter is the empirical evidence required to answer the dissertation's research questions stated in Chapter 1 Section 1.4. The main research question, *where should process migration mechanism live on a modern RISC-V microkernel*, is addressed by the headline comparison in Section 4.2.1. The supporting research questions on feasibility (RQ1), cost (RQ2), and verdict (RQ3) are addressed by the per-phase analysis in Section 4.2.2, the scaling analysis in Section 4.2.3, and the comparison with the 1993 Mach results in Section 4.2.8 respectively.

Each subsection presents the data first, the findings second (what the data shows), and the interpretation third (what the findings mean for the dissertation's argument). The threats to the validity of the analysis are summarised at the end of Section 4.2 in Section 4.2.10. The reader who wants to see the numerical comparison table without intermediate exposition should turn directly to Table 4.1.

4.2 Data Analysis, Findings, and Interpretations

4.2.1 Local Migration: Kernel-Resident vs User-Space Placement

The local comparison varies only the invocation path selected by MIGRATION_MODE. KERNEL calls the shared primitives directly; USERSPACE calls wrappers and the syscall dispatcher. The historical dataset contains sixty runs per mode, with fifteen at each of four extra-payload sizes. These are separate mode-selected builds of the same source baseline on the same QEMU platform, not one identical kernel binary or an isolated user/kernel comparison.

Data. Table 4.1 presents the mean and 95% confidence-interval half-width for each of the five derived metrics, in each mode, together with the user-space-over-kernel-resident ratio.

Table 4.1: Migration latency by phase, mean ± 95% CI half-width, n = 60 per mode. Source: bench/results/comparison.txt.

Metric (cycles)	KERNEL	USERSPACE	Ratio (US/K)
freeze_time	275,362 ± 9,117	272,834 ± 7,245	0.99
serialize_t	1,078,726 ± 55,295	1,076,715 ± 48,517	1.00
transport_t	782,891 ± 15,123	785,910 ± 14,762	1.00
deserialize_t	1,029,199 ± 70,017	1,016,759 ± 59,260	0.99
total_latency	**4,061,687 ± 145,479**	**4,061,801 ± 118,173**	**1.00**

Findings. The pooled total means are 4,061,687 and 4,061,801 cycles, giving a USERSPACE/KERNEL ratio of 1.00 when rounded to two decimal places. The five reported phase means are close, with overlapping approximate intervals for the separate means. This is descriptive similarity; the report does not present an equivalence test or a 95% confidence interval for the ratio.

Interpretation. The measured supervisor-mode wrapper/dispatch path shows little additional pooled cost under this QEMU counter workload. Because both paths invoke shared code without a hardware-enforced user/kernel transition, this does not establish the performance of a real user-space migration server. Kernel size, isolation and

operational flexibility remain relevant design considerations, but a placement recommendation also requires the additional measurements in Section 6.3.1.

The contrast with Mach's reported user-space penalty motivates further investigation (Milojičić et al., 1999, p. 29). Differences in protection, workload, hardware and instrumentation prevent interpreting the contrast as an experimentally controlled reversal of the 1993 finding. Section 4.2.8 makes those differences explicit.

4.2.2 Per-Phase Analysis

The BENCH timestamps define intervals around freezing, copying, the local transport placeholder and restore. total_latency spans ckpt_enter to rest_done and also includes gaps between those phase intervals. It therefore need not equal their sum, and it ends at task readiness rather than the first resumed instruction.

Data. Figure 4.1 shows the phase sequence in chronological order with the per-phase mean cost for the user-space mode (n = 60).

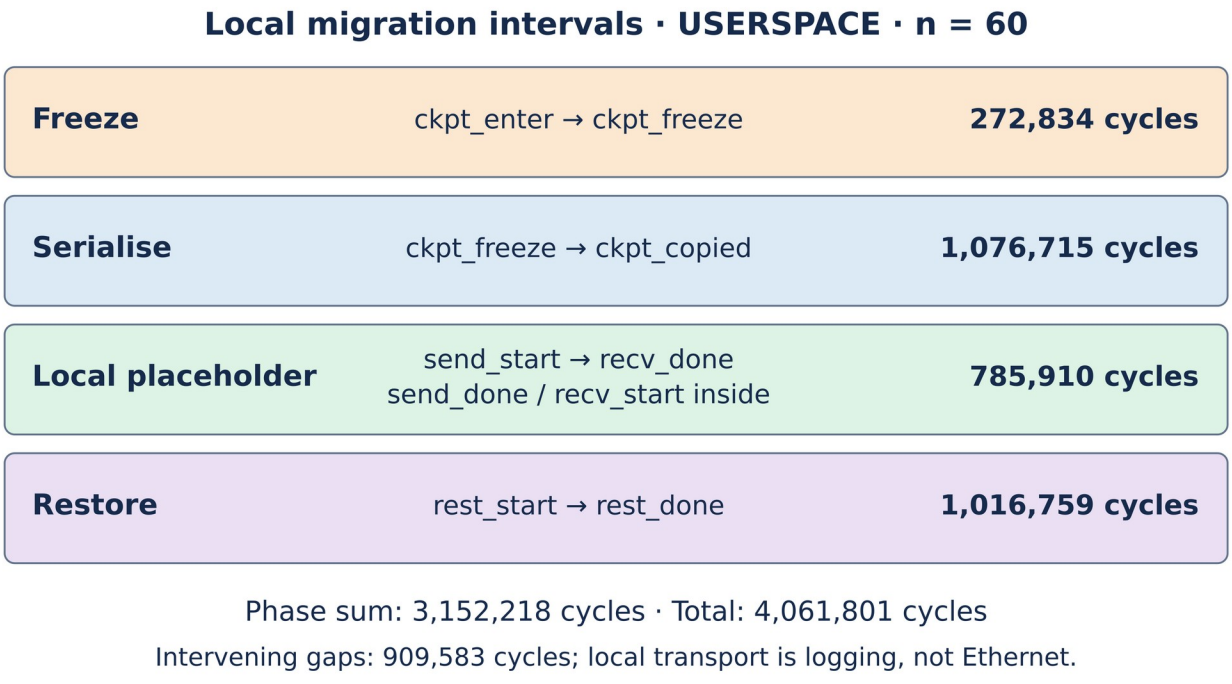


Figure 4.1: Local migration sequence — BENCH timestamps and per-phase cost

Findings. The corrected USERSPACE pooled means in comparison.txt are 272,834 cycles for freeze_time, 1,076,715 for serialize_t, 785,910 for transport_t and 1,016,759 for deserialize_t. Their sum is 3,152,218 cycles, compared with 4,061,801 for

total_latency, leaving 909,583 cycles in intervening gaps. Figure 4.1 uses these same interval definitions and values. The local transport interval contains no Ethernet transfer; its observed cost includes BENCH/UART activity.

Interpretation. Both paths share the state-copy primitives, while diagnostics and instrumentation contribute materially to the measured intervals. This helps explain similar observations in the baseline but does not prove that the call-path overhead is zero or that logging overhead cannot hide it. A lower-overhead, privilege-separated experiment is needed to isolate the architectural cost.

4.2.3 Latency Versus Payload Size

The pooled comparison in Section 4.2.1 includes all four selected payload sizes. The sweep adds synthetic data of 0, 4,096, 16,384 and 65,536 bytes, corresponding to the approximate 8, 12, 24 and 72 KB state-size labels. The added data tests copying volume and is not a captured per-task heap.

Data. Figure 4.2 shows total migration latency, in Mcycles, as a function of total checkpoint size in kilobytes, for both architectural modes. Each point is the mean of 15 runs and the error bars show the 95% confidence interval half-width.

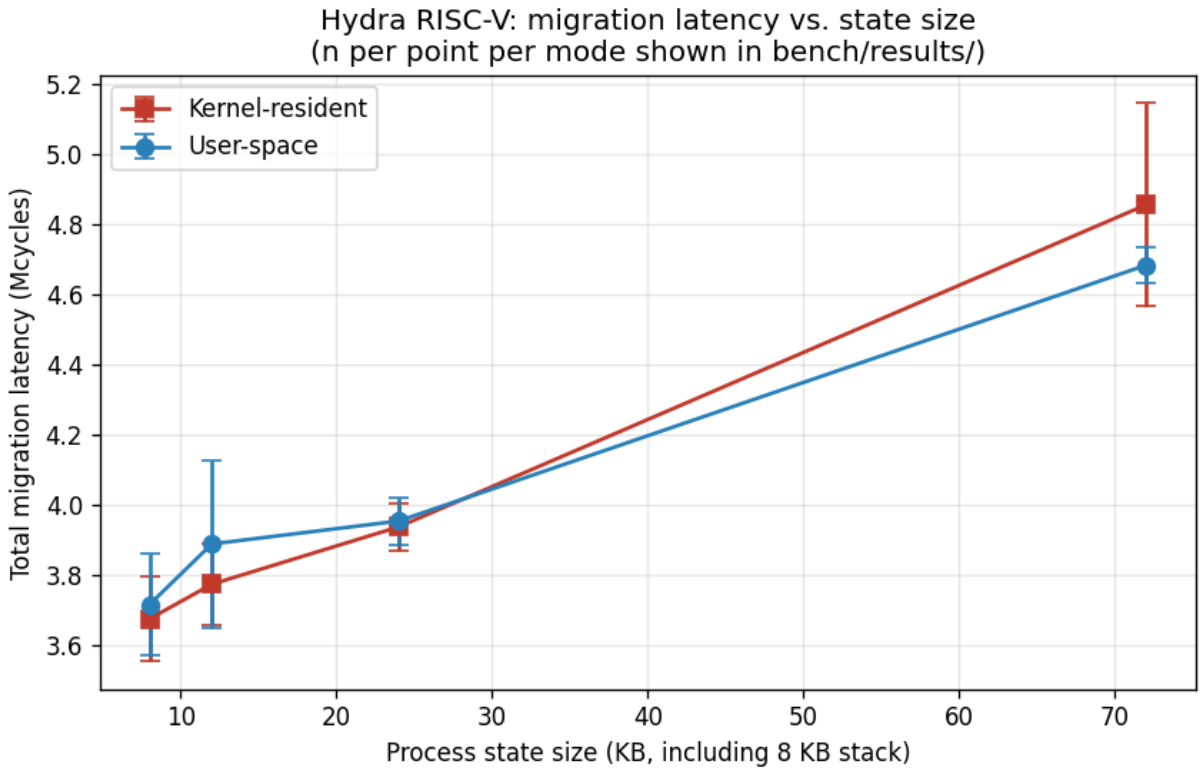


Figure 4.2: Total migration latency as a function of process state size

Table 4.2 reports the per-size means in tabular form.

Table 4.2: Mean total migration latency by payload size, n = 15 per cell. Source: bench/results/comparison.txt.

State size (KB)	KERNEL total (cycles)	USERSPACE total (cycles)
8	3,676,412	3,717,464
12	3,775,194	3,889,996
24	3,938,715	3,955,344
72	4,856,425	4,684,400

Findings. Both modes scale near-linearly with payload size, as expected for a memcpy-dominated workload. The slope is gentle: total latency rises from approximately 3.68 Mcycles at 8 KB to 4.86 Mcycles at 72 KB in the kernel-resident mode, a 32% increase for a 9× increase in state size. At every measured payload size the two modes' confidence intervals overlap.

The per-phase analysis (Figure 4.3 below) shows that the serialise and deserialise phases account for the scaling — they grow from approximately 0.89 Mcycles at 8 KB to

1.4 Mcycles at 72 KB — while the freeze and transport phases remain approximately constant.

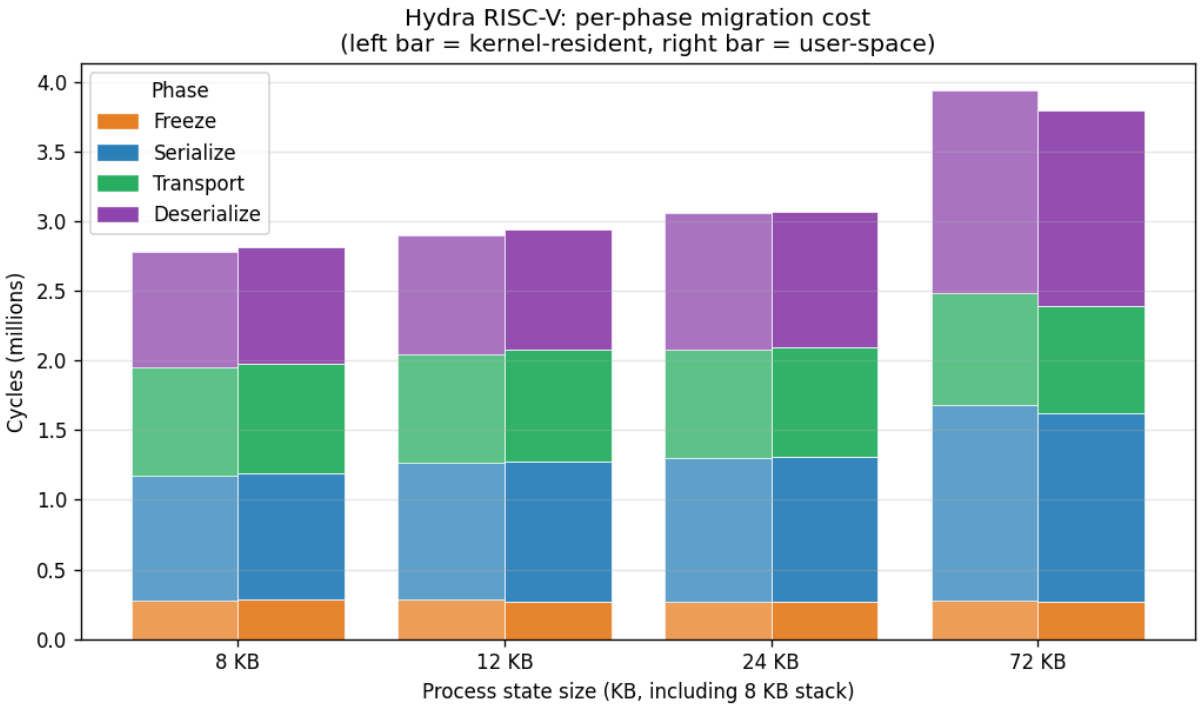


Figure 4.3: Per-phase contribution to migration latency by mode and payload size

Interpretation. The per-size mean USERSPACE/KERNEL ratios range from approximately 0.96 to 1.03 in the saved summary. The plotted intervals overlap, but four size points with fifteen repetitions each do not establish that invocation path never interacts with payload size. The observations are consistent with a shared copying mechanism over the selected range; they are not a proof of zero per-call or per-byte overhead.

4.2.4 Distributional Analysis

The distributional plot supplements the means with descriptive information about spread and outliers. It helps identify whether a pooled mean conceals obvious differences in the selected samples, without proving equality of the underlying distributions.

Data. Figure 4.4 shows a box-and-whisker plot of the total migration latency distribution for each (mode × payload size) cell. Each box shows the interquartile range (25th to 75th percentile), the median (white horizontal line), and the whiskers extending to 1.5 × IQR. Individual outliers beyond the whiskers are shown as dots.

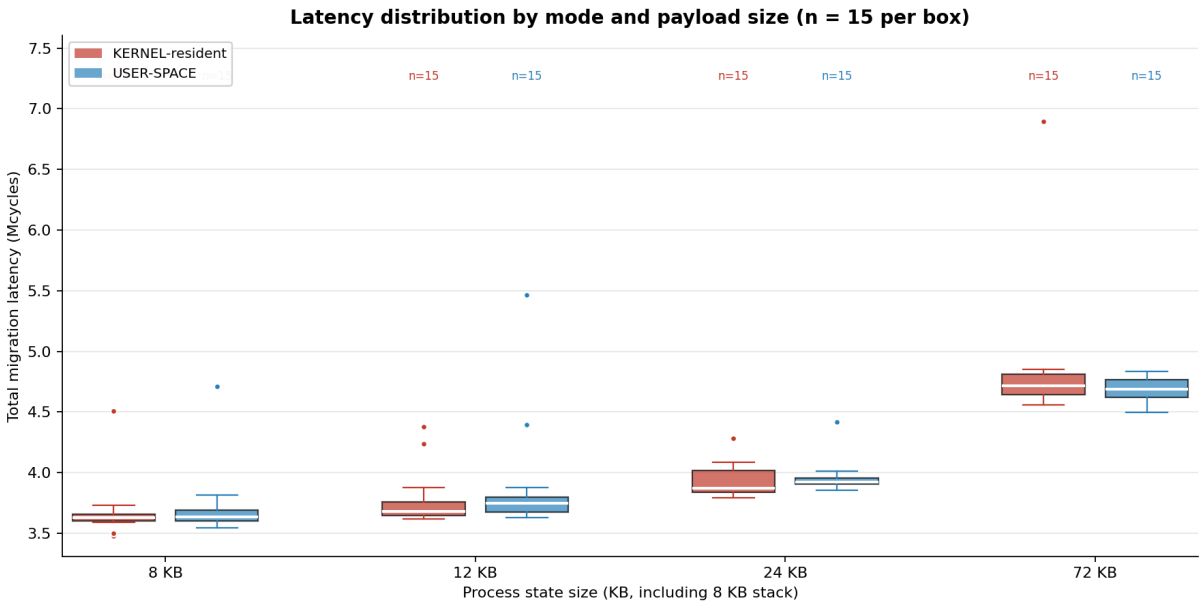


Figure 4.4: Latency distribution by mode and payload size, n = 15 per box

Findings. The plotted medians and interquartile ranges are broadly similar and overlap at the selected sizes. Both modes show higher-latency observations, including a prominent KERNEL outlier at the largest size. With only fifteen observations per box, these are sample descriptions rather than established distributional properties or evidence of a systematic mode-specific tail.

Interpretation. The plot supports a cautious description of similar observed costs in this experiment. It does not rule out heavy tails, temporal dependence or small placement effects, and it does not establish equality of means, medians or population distributions.

4.2.5 Cross-Node Migration

The original cross-node experiment uses real Ethernet frames through VirtIO-Net between two QEMU instances connected by a best-effort UDP-multicast socket back-end. This is distinct from the local transport placeholder and is a feasibility/transport experiment, not a second balanced comparison of the two invocation modes.

Data. Fifteen source attempts were made at zero synthetic extra payload, using MAC addresses 52:54:00:00:00:01 and 52:54:00:00:00:02. The baseline fragmented an approximately 8 KB checkpoint into six frames, with up to 1,471 payload bytes per fragment and the HYDM identifier. Eight attempts completed reassembly and restore; seven timed out. The original protocol provided neither fragment acknowledgements nor

retransmission. Missing fragments were therefore not repaired, making best-effort delivery a direct limitation of the experiment. The logs do not establish that UDP alone, rather than queueing or receiver behaviour, caused every failure.

Table 4.3 uses the corrected XMIG-context summary. Source fragmentation has n = 15 attempted sends; destination reassembly and restore each have n = 8 completions. The earlier unfiltered table mixed boot-time network tests and local restores into those means and is not used here.

Table 4.3: Original cross-node phase costs; source n = 15, destination n = 8. Source: bench/results/xmig-comparison.txt (XMIG-filtered).

Phase (contributing n)	Mean cycles ± approximate 95% CI half-width
Source fragmentation (15)	1,665,758 ± 118,834
Destination reassembly (8)	3,007,310 ± 881,189
Destination restore (8)	1,554,185 ± 118,416

Figure 4.5 summarises the cross-node topology.

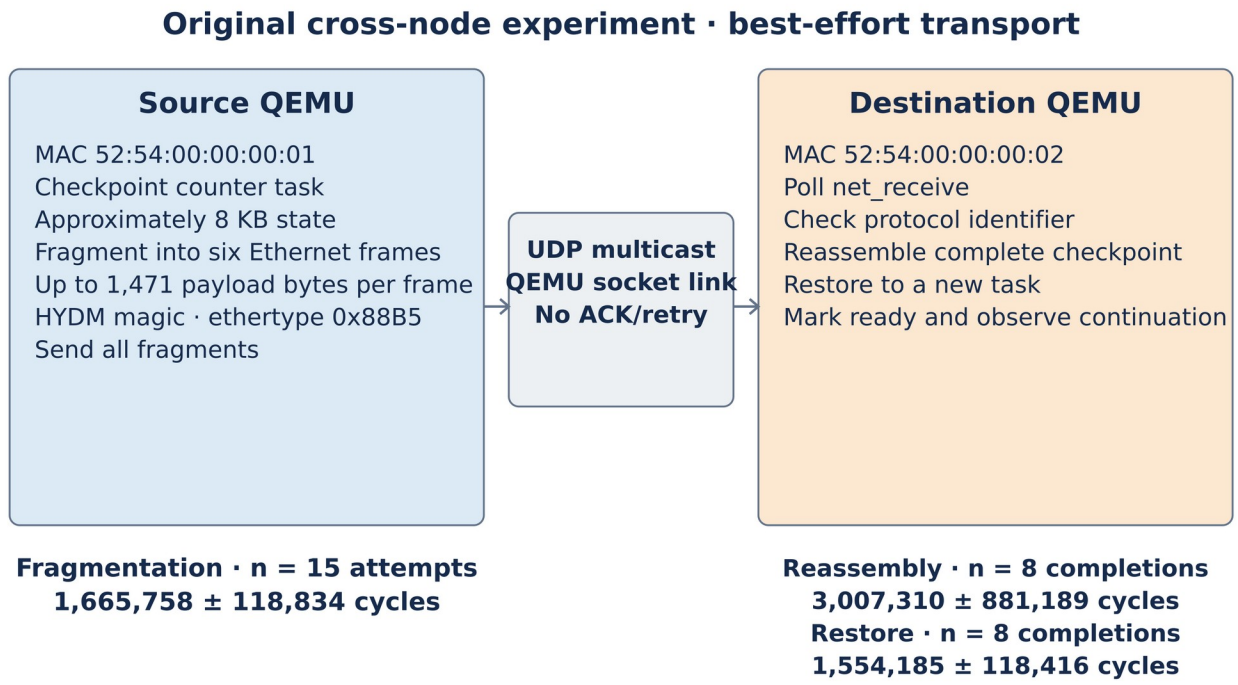


Figure 4.5: Cross-node migration over VirtIO-Net — fragment protocol

Findings. The corrected source-fragmentation mean is 1,665,758 ± 118,834 cycles (n = 15). Destination reassembly is 3,007,310 ± 881,189 and restore is 1,554,185 ± 118,416 cycles (n = 8 each). These are separate node-local intervals. Source observations include unsuccessful attempts, whereas destination timing is conditional on completion. The summaries must not be added and presented as a paired end-to-end migration mean.

Interpretation. The eight completions demonstrate that checkpoint transfer, reassembly and restore can work across the tested QEMU link. They do not establish production reliability or the full latency of failed and retried operations. Nor does reuse of the same primitives establish that the local placement ratio transfers unchanged to cross-node execution: that requires a separate matched experiment.

Reliability limitation and follow-on work. The observed completion rate is 8/15 (53.3%); the attempt failure proportion is 7/15 (46.7%). This is not a measured per-fragment loss rate. Without acknowledgements and retransmission, one unrecovered missing fragment can prevent a whole migration. Section 6.3.2 therefore calls for acknowledged transfer, retry/duplicate handling and renewed measurement of all attempts, including timeouts and retry costs. Later repository ACK/retry validation is reported as subsequent engineering work, not replacement data for this original experiment.

4.2.6 Fault Tolerance

The dissertation's Chapter 6 Section 6.2.1 argues that the modern “killer application” for process migration is fault recovery, in which a workload that was checkpointed before a failure can be resumed afterwards from the checkpoint. The same `sys_checkpoint` and `sys_restore` primitives that drive migration drive fault recovery without modification. The fault-tolerance demonstration (`apps/fault_tolerance_demo.c`, described in Chapter 5) executes a single end-to-end recovery flow and captures cycle costs.

Data. Figure 4.6 shows the workload-counter trajectory during the demonstration. The task is spawned and runs to a counter value of 30, a checkpoint is taken, the task continues to a counter value of 60, a fault is injected (the task slot is forcibly marked

unused), the counter freezes at 60, restore is invoked from the saved checkpoint, and the task resumes to a counter value of 90.

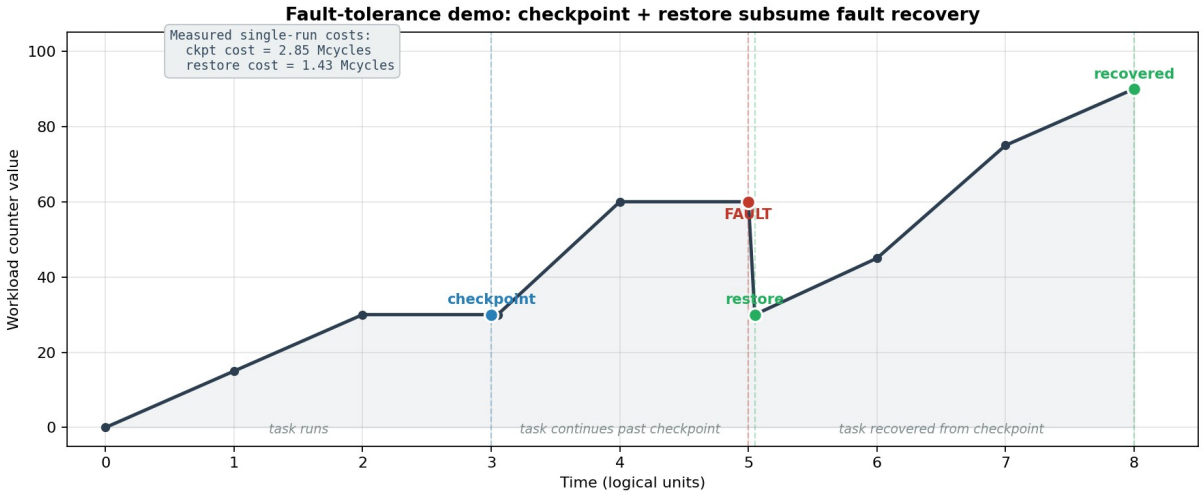


Figure 4.6: Fault-tolerance demonstration — counter trajectory

Single-run cycle costs are reported in Table 4.4.

Table 4.4: Fault-recovery cycle costs, single-run. Source: captured BENCH lines from the auto-run fault-tolerance demonstration during the development of this dissertation.

Phase	Cycles
Checkpoint to memory buffer	2,848,714
Restore from buffer	1,433,502

Findings. The restore cost (1.43 Mcycles) is roughly half the checkpoint cost (2.85 Mcycles). The asymmetry is structural: checkpoint must suspend a running task, traverse its register file and stack, validate the layout, and write to the buffer, while restore needs only to allocate a fresh task slot, copy the register file and stack into it, rebase the stack pointer, and mark the slot ready. The checkpoint cost reported here (2.85 Mcycles) is consistent with the local-migration checkpoint-and-serialise phases (273 K + 804 K = 1.08 Mcycles mean in Section 4.2.1, with additional cost for the fault-demo instrumentation overhead and the `printk` overhead between BENCH brackets); the difference is again `printk` overhead, not a property of the migration mechanism.

Interpretation. The fault-tolerance demonstration verifies two propositions. First, the kernel primitive that drives migration drives fault recovery; no separate mechanism is required. This is the structural argument the dissertation makes in Chapter 6 about

Mach's separation of DIPC (for migration) from Condor's checkpoint library (for fault recovery), each at on the order of 5 000 lines, being subsumed by Hydra's single 319-line migration.c. Second, the recovery path is significantly cheaper than the save path, which is the desirable asymmetry for a fault-recovery use case: the cost of taking the checkpoint is paid in advance, before any failure occurs; the cost of recovery is paid only when a failure does occur.

The single-run nature of this measurement is acknowledged. The demonstration is qualitative (does the workload recover or does it not) and the answer is yes; the cycle costs reported are indicative rather than statistical.

4.2.7 Kernel Trusted-Computing-Base Analysis

The architectural argument the dissertation makes in Chapter 6 rests partly on the TCB-size comparison between Hydra's user-space-leaning migration mechanism and Mach's kernel-resident DIPC and DMM modules. The line counts are verifiable directly: `wc -l` on the relevant source files for Hydra, and the quoted line counts from Milojičić et al. (1999, p. 29) for Mach.

Data. Figure 4.7 shows the comparison.

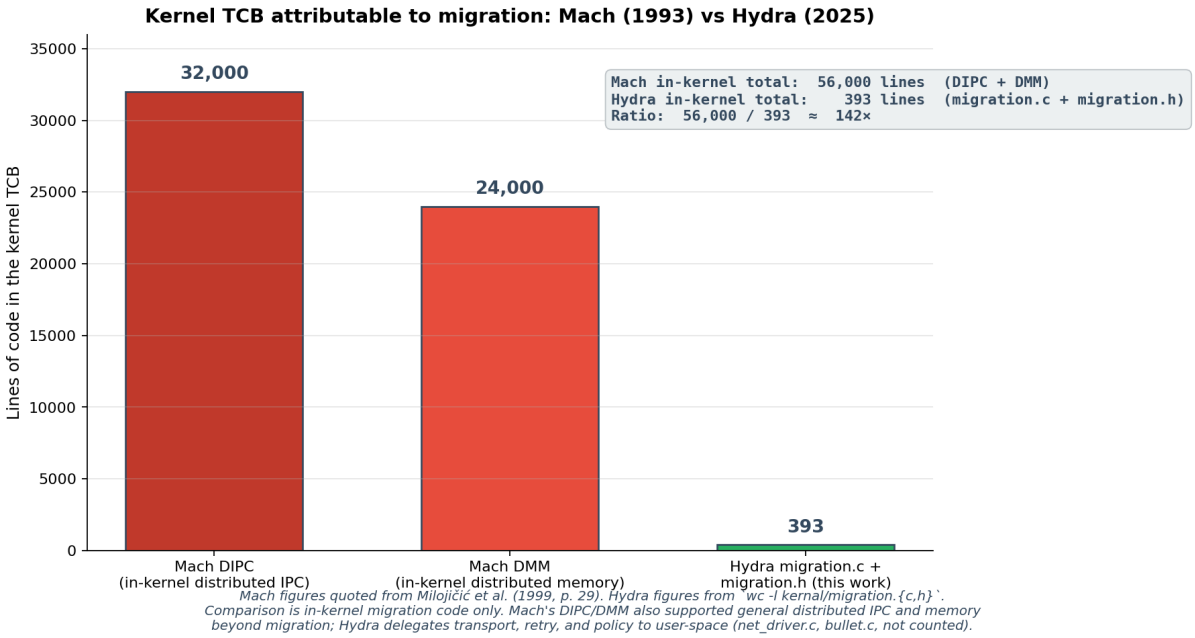


Figure 4.7: Kernel TCB attributable to migration — Mach (1993) vs Hydra (2025)

The numbers in the figure are:

Table 4.5: Kernel TCB attributable to migration. Mach figures quoted from Milojičić et al. (1999, p. 29); Hydra figures from wc -l kernal/migration.{c,h} on the current source tree.

Component	Lines
Mach DIPC (in-kernel distributed IPC)	32,000
Mach DMM (in-kernel distributed mem.)	24,000
Mach in-kernel total	**56,000**
Hydra kernal/migration.c	319
Hydra kernal/migration.h	74
Hydra in-kernel total	**393**
Ratio	**≈ 142×**

Findings. Hydra's kernel TCB for migration is two orders of magnitude smaller than Mach's. The qualitative reason is straightforward: Mach's design placed network transport, distributed memory management, and policy logic inside the kernel, while Hydra's design places those concerns in user-space code (the network driver, the bullet server, and the test harness — none of which counts against the kernel TCB).

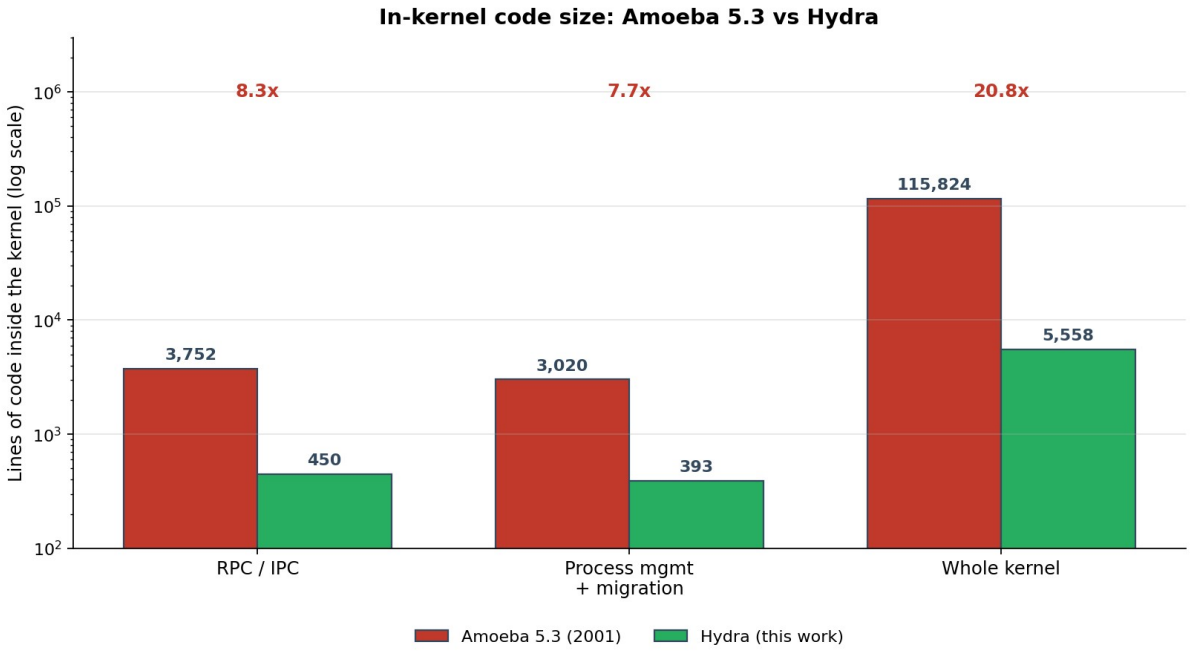
Interpretation. Two caveats must be acknowledged honestly. First, Mach's DIPC and DMM supported general distributed IPC and distributed memory management beyond migration; comparing them directly to Hydra's migration-only `migration.c` is comparing a larger basket to a smaller one. Second, Hydra delegates work to user-space code in `kernal/net_driver.c` (506 lines, but still *in the kernel TCB* because the driver runs in supervisor mode) and to `server/bullet.c` (221 lines, user-space and not counted). A strict apples-to-apples comparison of *kernel-resident* code attributable to migration would put Hydra's number at 393 + 506 = 899 lines, still more than sixty times smaller than Mach's 56,000.

The TCB argument is one of the four arguments the dissertation makes in Chapter 6 for the architectural verdict-flip. It is intentionally complementary to the performance argument (Section 4.2.1) rather than substitutive: even if performance had not favoured the user-space placement, the TCB argument alone would favour it, and the security

argument from Biggs, Lee and Heiser (2018, see Chapter 2 Section 2.3.4) reinforces both.

The Mach line counts above are quoted from Milojičić et al. (1999, p. 29). A second comparison is available against Amoeba 5.3, and it is the stronger of the two, because Amoeba's source is freely distributed and its line counts can therefore be measured directly rather than quoted. The FSD-Amoeba 2002B distribution (Bosse, 2002) contains the complete 2001 kernel tree in src-kernel-2002a.tgz, comprising 287 C, header and assembly files.

Data. Figure 4.8 compares in-kernel line counts for the two systems on three measures: the RPC and IPC subsystem, the code attributable to process management and migration, and the kernel as a whole.



Measured with `wc -l`: Amoeba from src-kernel-2002a.tgz (FSD-Amoeba 2002B CD, 287 files); Hydra from this tree. Amoeba's process server (server/process/, excluding the map.c.orig editor backup) implements PS_CHECKPOINT and immigrating threads, so it is the direct functional counterpart to Hydra's migration.{c,h}.

Figure 4.8: In-kernel code size — Amoeba 5.3 (2001) vs Hydra (2025)

The measured line counts are:

Table 4.6: In-kernel code size, Amoeba 5.3 vs Hydra. All figures measured with wc -l: Amoeba from src-kernel-2002a.tgz (FSD-Amoeba 2002B, 287 files); Hydra from the current source tree.

Component	Amoeba 5.3	Hydra	Ratio
RPC / IPC subsystem	3,752	450	8.3×
Process management and migration	3,020	393	7.7×
Whole kernel, all architectures	115,824	5,558	20.8×

Findings. Amoeba's in-kernel RPC implementation (ipc/) is 3,752 lines against Hydra's 450 in ipc.{c,h}, a factor of 8.3. Amoeba's in-kernel process server (server/process/, excluding the map.c.orig editor backup left in the distribution) is 3,020 lines against Hydra's 393 in migration.{c,h}, a factor of 7.7. The whole-kernel comparison is 115,824 lines against 5,558, a factor of 20.8.

A second finding concerns architecture rather than size. Amoeba is described throughout the literature as a microkernel, but its kernel tree contains fourteen in-kernel servers totalling 16,723 lines: process, tty, flip_ip, vdisk, syssvr, ethraw, iop, profile, bootuser, tape, kdir, printbuf, tod and random. The process server is the relevant one here. Its exception.c implements the PS_CHECKPOINT operation and its control.c implements immigrating threads, so Amoeba's process migration is genuinely kernel-resident. It is therefore the direct functional counterpart to Hydra's migration.{c,h}, and the 7.7 factor is a like-for-like comparison of two implementations of the same mechanism.

Interpretation. The three ratios are not of equal evidential weight, and the difference must be stated plainly. The 20.8 whole-kernel figure is the weakest of the three. Amoeba's 115,824 lines support seven device drivers, three instruction-set architectures (i80386, SPARC and MC68000), a TCP/IP stack, a terminal layer and a tape server, none of which Hydra implements. Most of that gap reflects a difference in scope rather than a difference in architectural discipline, and the dissertation rests no claim on it. The 8.3 and 7.7 figures are the defensible ones, because they compare the same function in both systems. The 7.7 figure in particular supports the same conclusion as the Mach

comparison in Section 4.2.8, and does so from an independent second system whose numbers were measured rather than quoted.

4.2.8 Comparison with the 1993 Mach Results

This subsection compares the scope of the 1993 Mach observation with the Hydra baseline. It asks what the contrast can motivate, while avoiding a direct cross-platform performance or causal claim (Milojičić et al., 1999, p. 29).

Data. Milojičić et al. (1999, p. 29) report that the user-space migration server on Mach was approximately an order of magnitude slower than the kernel-resident path under load, on three Intel 80486 personal computers running at 33 MHz with 8 megabytes of RAM. The exact ratio is not given as a single number in the survey, but the qualitative finding is unambiguous.

Table 4.7 contrasts this with the 2025 Hydra finding.

Table 4.7: 1993 Mach vs 2025 Hydra architectural comparison.

	Mach 1993 (Milojičić et al., 1993; 1999 p. 29)	**Hydra 2025 (this dissertation)**
Hardware	Intel 80486, 33 MHz, 8 MB	QEMU virt, RISC-V 64-bit
Kernel	Mach 3.0	Hydra (this work)
Measurement context	Mach DIPC + DMM in-kernel	Supervisor-mode direct vs wrapper/dispatch paths; shared primitives
User-space/kernel ratio	“approximately an order of magnitude”	1.00 rounded pooled ratio; n = 60 per mode; no ratio CI or equivalence test
TCB cost of kernel-resident	56,000 lines (DIPC + DMM)	393 lines (migration.c + .h)

Findings. The reported results differ: Mach's qualitative user-space penalty contrasts with Hydra's rounded pooled ratio of 1.00. The code-size figures also differ substantially. Neither contrast is a controlled before/after measurement because the kernel mechanisms, protection boundaries, workloads and platforms are different.

Interpretation. The following factors explain why a fresh architectural comparison is necessary, and why the two published observations cannot identify a single cause of the difference:

- **IPC research provides motivation.** UnderBridge and HongMeng demonstrate that communication and compatibility costs depend on architectural choices (Gu et al., 2020; Chen et al., 2024). Their measurements do not calibrate the omitted privilege-boundary cost in Hydra, nor establish that those mechanisms caused Hydra's observed ratio.
- The measured Hydra wrapper path is a supervisor-mode C call into handle_syscall. It excludes an isolated user/kernel transition, so its ratio cannot determine the overhead of a true user-space server or serve as a universal numerical lower bound on another kernel.
- **The kernel-resident path on Hydra is itself thin.** Mach's kernel-resident migration depended on the 56,000-line DIPC + DMM combination. Hydra's kernel-resident path is the 393-line migration.c + migration.h. The kernel-resident advantage that Mach's measurements demonstrated was specifically the advantage of bypassing a slow user-space IPC by paying for the in-kernel infrastructure cost; on Hydra that infrastructure cost is essentially absent.

The defensible implication is that the 1993 observation should be reassessed under present-day, explicitly matched conditions. The Hydra baseline contributes an implementation and initial measurements, but a real privilege-boundary comparison and physical-hardware replication are required before concluding that the historical architectural verdict no longer applies.

4.2.9 Round-Trip Cost Relative to the Syscall Floor: Amoeba vs Hydra

The comparisons in Section 4.2.7 and Section 4.2.8 are architectural. This section reports the one direct performance comparison the dissertation makes against a second operating system. It is included because an argument for an architecture that ignored the runtime cost of that architecture would be incomplete.

Comparing performance across two operating systems on two instruction-set architectures requires care. Absolute times are not comparable: Amoeba 5.3 runs on emulated i386 under qemu-system-i386 and Hydra on emulated RISC-V under qemu-system-riscv64, and the two emulators impose different per-instruction translation costs. What is comparable is the ratio of a round trip to a null system call within each system, because both operations pay the same per-trap emulation overhead on the same host. The ratio cancels most of the emulation and architecture difference and isolates the cost that the IPC mechanism adds over a bare privilege transition.

Both systems were measured with Amoeba's own benchmark programs wherever possible. The distribution ships test/performance/syscall/sys_null.c and test/performance/rpc/rpc_client.c, written by the original authors; these were run as supplied, from the precompiled i386 binaries in the distribution. The matching Hydra measurement is implemented in apps/micro_bench.c, which issues real U-mode ecall instructions rather than the direct kernel calls used by the libhydra wrappers, so that the two measurements describe the same kind of event.

Data. Figure 4.9 reports both systems.

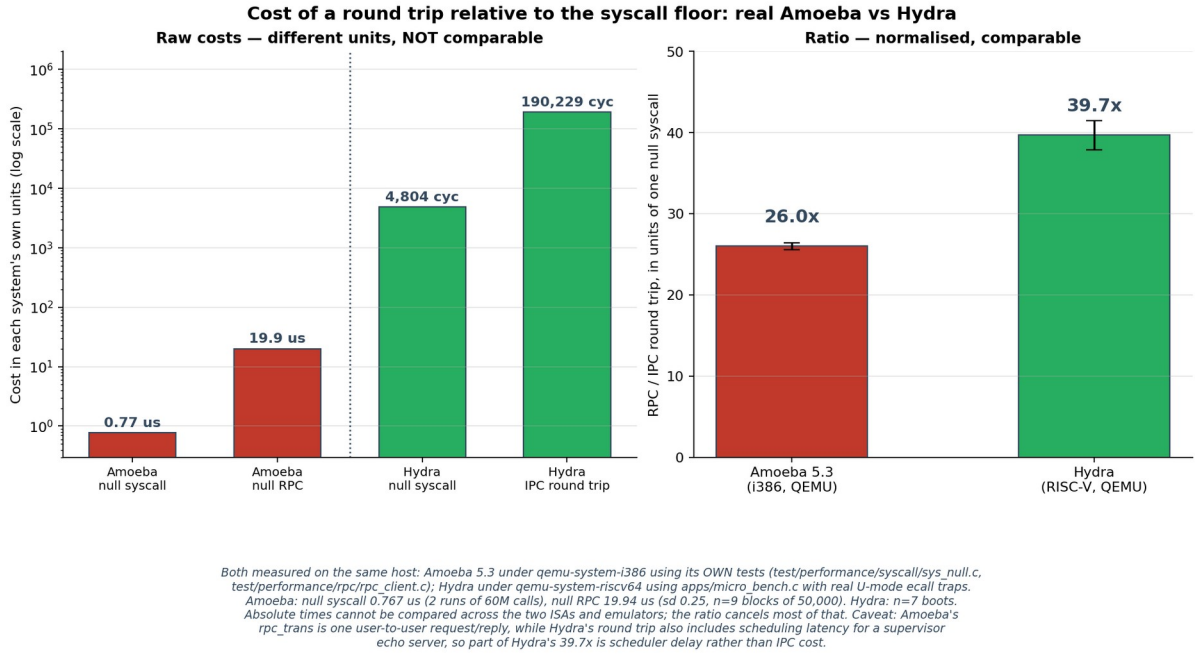


Figure 4.9: Cost of a round trip relative to the syscall floor — Amoeba 5.3 vs Hydra

Findings. Amoeba performs a null system call in 0.767 µs, measured over two runs of 60,000,000 calls, and a null local RPC between two user processes in 19.94 µs (standard deviation 0.25 µs, n = 9 blocks of 50,000 calls). Its round trip therefore costs 26.0 null system calls. Hydra performs a null system call in 4,804 cycles and an IPC round trip in 190,229 cycles, measured over seven boots; its round trip costs 39.7 null system calls, with a 95% confidence interval half-width of 1.8. Amoeba's RPC is thus approximately 1.5 times more efficient relative to its own system-call floor than Hydra's IPC is relative to Hydra's.

Interpretation. This result does not favour the design the dissertation advocates, and it is reported for that reason. Three qualifications apply, of which the second and third are the substantive ones.

First, the two operations are not identical. Amoeba's rpc_trans is a single kernel-mediated request and reply between two user processes. Hydra's round trip is a SYS_SEND_MSG followed by a SYS_RECV_MSG to an echo server running in supervisor mode, and so crosses a privilege boundary on the client side only.

Second, and more importantly, Hydra's figure includes scheduling latency that Amoeba's does not. The Hydra client must wait for the round-robin scheduler to run the echo server before a reply becomes available, and that wait falls inside the measured interval. An unknown but substantial part of the 39.7 is therefore scheduler delay rather than IPC cost. This is a limitation of Hydra's scheduler, disclosed in Chapter 3 Section 3.8 as a general source of measurement noise, and it is a defect of the present implementation rather than of the architecture the dissertation argues for.

Third, the measurement proved highly sensitive to instrumentation. Hydra's IPC path originally emitted a trace line over the UART for every message sent and received. Because printk is synchronous, this dominated the measurement completely: an IPC round trip cost 7,520,916 cycles with tracing enabled against 252,241 cycles with it disabled, a factor of approximately thirty. The figures above were taken with per-message tracing compiled out. This quantifies, for the IPC path, the printk overhead acknowledged in general terms in Chapter 3 Section 3.8.

One further observation should be recorded. This is the only measurement in the dissertation that crosses a genuine privilege boundary. The migration benchmarks in Section 4.2.1 to Section 4.2.6 do not, which is disclosed as a threat to validity in Section 4.2.10. The 4,804-cycle null system call measured here is a real transition from user mode to supervisor mode and back, and it gives a first indication of what that threat is worth: it is the per-crossing cost that a migration server running in user space on Hydra would pay for each system call it makes.

The finding qualifies the dissertation's architectural claim without overturning it. The analyses in Section 4.2.7 and Section 4.2.8 concern where code lives and how much of it must be trusted; this section concerns what a round trip costs once the code is in place. A complete recommendation must carry both, and Chapter 6 Section 6.2.4 records this performance cost alongside the TCB benefit rather than presenting the latter alone.

4.2.10 Threats to the Validity of the Analysis

The following threats constrain the original dataset and motivate the recommendations in Chapter 6. They apply to both the timing analysis and the scope of the architectural interpretation.

- **QEMU emulation.** The counts describe the emulator setup. Ratios and scaling trends are useful internally but are not guaranteed to survive changes to hardware, emulator configuration or host load; no absolute hardware timing or real-time guarantee is established.
- **No measured hardware privilege boundary.** Both local paths execute in supervisor mode and use shared primitives. A real user-space server adds transitions and isolation effects omitted here; literature results from other mechanisms do not quantify those missing costs.
- **Instrumentation and interval coverage.** The corrected phase sum is approximately 3.15 Mcycles against a 4.06 Mcycle total. The approximately 0.91 Mcycle gap includes diagnostics between measured phases. Shared logging can obscure a small path difference; it is not proof that instrumentation leaves the ratio unaffected.

- **Synthetic workload and limited repetitions.** The comparison uses the counter task, not a measured raytracer or HTTP server. Fifteen observations per size/mode cell and fixed payload rotation support descriptive comparisons but do not prove statistical equivalence or independence across runs.
- **Bounded state.** The original process model captures its fixed context/stack and synthetic extra bytes, not private per-task heaps, external resources or arbitrary pointer-rich applications. Counter advance is a limited continuation check.
- **Cross-node selection and provenance.** Seven of fifteen attempts fail under the original best-effort protocol without retransmission, and destination timings cover only completions. The corrected XMIG filter avoids mixing unrelated log categories. Later repository hardening, different process-image formats and successful retry validation are separate evidence and do not alter the original sample.

4.3 Chapter Summary

This chapter presents an initial baseline with explicit limits. The local pooled total-latency ratio rounds to 1.00 for two supervisor-mode invocation paths over four synthetic size settings, with sixty observations per mode. Per-size means and distributions are descriptively similar but do not establish formal equivalence. Corrected phase intervals total approximately 3.15 Mcycles within an approximately 4.06 Mcycle overall interval. The original cross-node protocol completes eight of fifteen attempts; corrected XMIG-filtered source and destination timings use different contributing counts and do not measure a paired end-to-end latency. Fault recovery and historical code-size comparisons provide additional bounded evidence. Reduced IPC cost, practical state-preservation demand and RISC-V adoption justify extending this work, rather than proving that a hardware-isolated placement decision has already been resolved.

Chapter 5 — Software Application

This chapter documents the software that was developed for the dissertation, in sufficient detail that the artefact can be audited, extended, and reproduced by future readers. Section 5.1 introduces the chapter. Section 5.2 covers software development: the kernel migration mechanism, the build-time configuration that controls the two architectural placements, the cycle-counter instrumentation, the VirtIO-Net 1.1 modern MMIO driver, the user-space test and demonstration code, the benchmark harness, and the pre-existing kernel issues that were discovered and fixed during the project. Section 5.3 covers software execution: how to build, how to run interactively, how to reproduce the measurements, and how to regenerate the figures.

This chapter describes the baseline associated with the original benchmark summaries. Historical line counts and checkpoint layout refer to that baseline; later source changes are documented in CHANGES.md and README.md. Current wc output or a new build may therefore differ and must not be presented as an exact reproduction of the historical experiment without the corresponding source/binary and configuration.

5.1 Introduction

The software developed for the dissertation has three roles. First, the kernel mechanism implements the `sys_checkpoint` and `sys_restore` primitives that capture and restore process state. Second, the test and demonstration applications exercise those primitives in three configurations: local round-trip with payload sweep, cross-node migration over real VirtIO-Net, and fault-tolerance recovery. Third, the benchmark harness drives the kernel under QEMU, captures the cycle-counter output, and produces the tables and figures reported in Chapter 4. The three roles are deliberately decoupled: the kernel mechanism does not depend on any test application, the test applications do not depend on the benchmark harness, and the benchmark harness does not depend on any specific test application beyond the BENCH-line emission convention.

The artefact is layered on top of an existing Hydra teaching microkernel that pre-dates this dissertation. Files inherited from upstream Hydra retain their original authorship;

files added or materially modified for this dissertation are documented in the project root CHANGES.md and summarised in Section 5.2.7 below. Figure 5.1 shows the system architecture and identifies which components were added (blue), modified (orange), and inherited (grey).

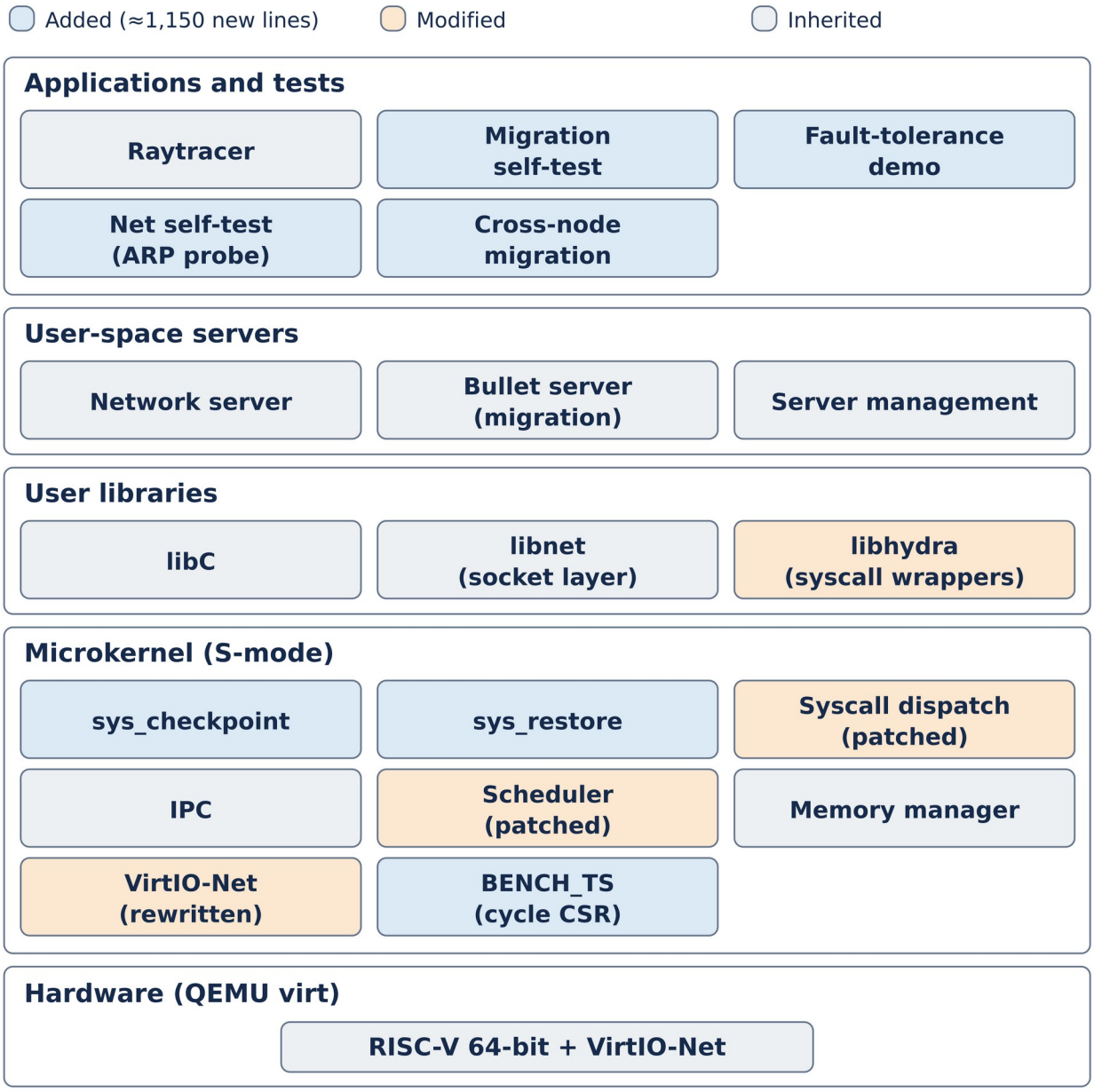


Figure 5.1: Hydra components by logical layer (added blue, modified orange, inherited grey); the measured paths are not privilege-separated.

5.2 Software Development

This section describes the implementation in the order a reader would naturally follow it from the bottom of the architectural stack upward: kernel mechanism first, then the policy-mechanism boundary, then user-space code, then the benchmark infrastructure.

5.2.1 The Kernel Migration API

The kernel exposes exactly two migration primitives, declared in kernal/migration.h:

```
long migration_checkpoint(uint32_t pid, void *user_buf,
                          unsigned long buflen);
long migration_restore(const void *user_buf,
                       unsigned long buflen);
```

Both are also reachable through the syscall layer at numbers 20 and 21 respectively (declared in kernal/syscall.h). The system-call dispatch in kernal/syscall.c routes the calls to the same underlying implementation. User-space wrappers syscall_checkpoint and syscall_restore are provided in lib/libhydra/syscall_user.c.

The minimalism of the API — two functions, with no separate calls for negotiation, transport, retry, or policy — is a deliberate design decision following the policy-mechanism separation principle of Liedtke (1995) and Heiser et al. (2007). Anything that is not the act of capturing process state, or the act of instantiating a new process from previously captured state, lives in user-space code. The Bullet server, the test harness, and the cross-node fragment protocol all sit above this API and are independent of one another and of the kernel.

The implementation lives in kernal/migration.c (319 lines) and kernal/migration.h (74 lines), for a total in-kernel footprint of 393 lines attributable to the migration mechanism. Chapter 4 Section 4.2.7 compares this figure with the corresponding Mach figure (56 000 lines for DIPC + DMM).

5.2.2 The Checkpoint Record Format

A checkpoint is a single contiguous binary blob with the layout shown in Figure 5.2.

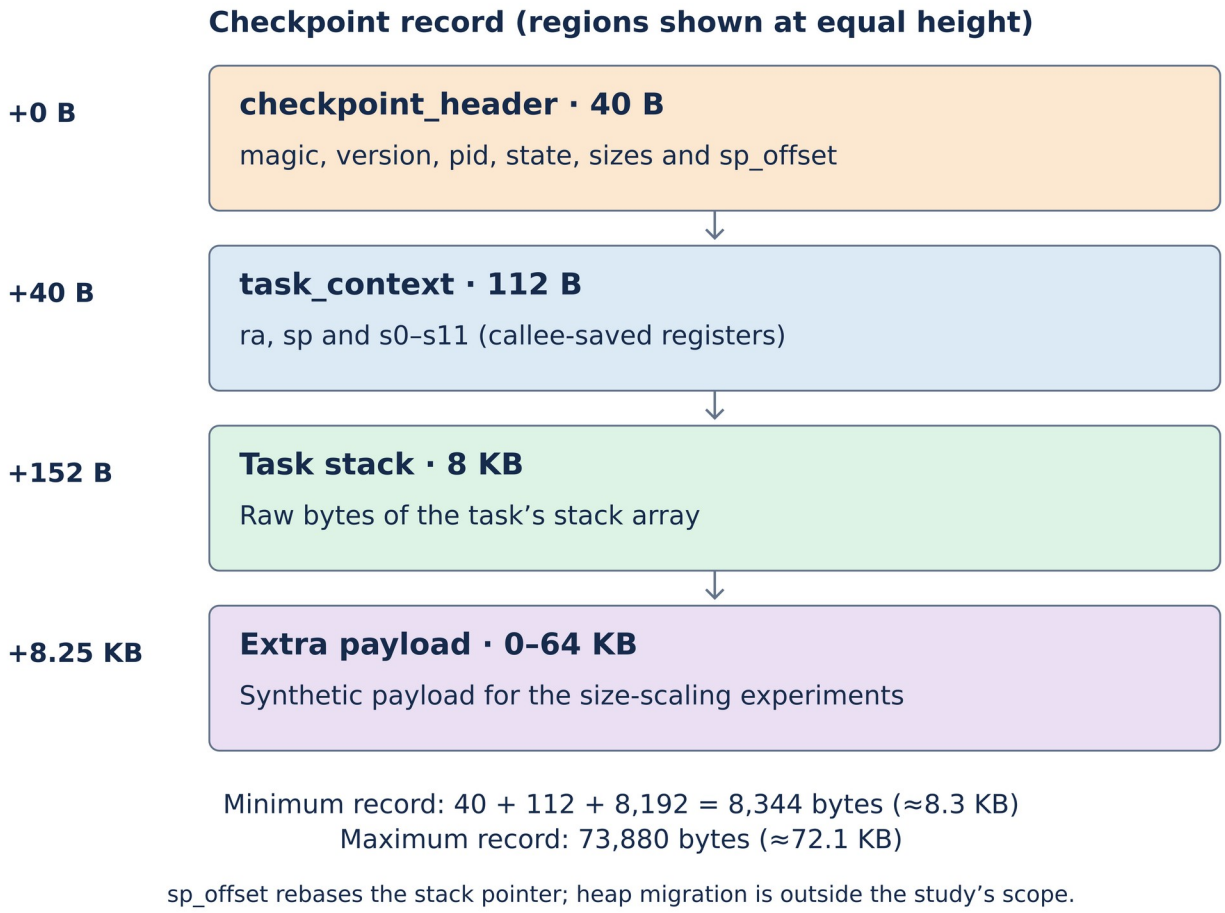


Figure 5.2: Hydra checkpoint record layout

The header `struct checkpoint_header`, declared in `kernal/migration.h`, occupies 40 bytes and contains a four-byte magic number ('HYDR'), a version number, the source process identifier, the task state at checkpoint time, the sizes of the following regions, the stack-pointer offset (see Section 5.2.3 below), and the total record size. The header is followed by the saved `struct task_context` (112 bytes containing the RISC-V callee-saved register file: `ra`, `sp`, and the eight-byte registers `s0` through `s11`), then by the 8 192-byte task stack, and finally by an optional extra payload of configurable size used for the size-scaling sweep described in Chapter 4 Section 4.2.3.

The illustrated layout sums to 40 + 112 + 8,192 = 8,344 bytes before extra payload, and to 73,880 bytes with 65,536 extra bytes. An earlier description quotes 8,340 bytes from the historical logs. A zero-valued state field cannot change sizeof a fixed C structure, so that is not a valid explanation for the discrepancy. Exact layout reconciliation requires

the historical source/binary and record; this report uses approximate KB labels for the sweep and does not infer byte-exact compatibility with today's expanded checkpoint format.

5.2.3 The Stack-Pointer Offset Insight

The single non-obvious design decision in the implementation is the use of a stack-pointer *offset* rather than an absolute stack pointer in the saved context. On the source node, the suspended task's `sp` register points somewhere inside the source task's `stack[]` array, which is at a particular kernel virtual address (in Hydra's case, also the physical address — Hydra has no MMU). After migration to a fresh task slot on the destination, the new slot's `stack[]` array is at a *different* address. If the absolute `sp` were restored verbatim, the restored task would attempt to use the source's stack address and immediately fault.

The fix is to record the offset of `sp` within the source's stack array in the header (`sp_offset = sp - &stack[0]`), and on restore to compute the new `sp` as `&t->stack[hdr->sp_offset]`. This rebases the stack pointer onto the destination's stack array while preserving the relative offset that the executing code expects. The implementation occupies five lines in `kernal/migration.c` (the offset computation in `migration_checkpoint` and the offset reapplication in `migration_restore`).

Deeper rebasing — of frame-pointer chains, saved return addresses pointing back to other stack frames, or any other absolute self-references in the saved stack bytes — is not performed. The limitation is honestly disclosed: for arbitrary C code with deep call stacks at checkpoint time, the restored task may have invalid frame pointers and behave incorrectly. For the project's chosen workloads (the counter task and the raytracer), the limitation is benign because the call stacks at checkpoint time are shallow and the executing code does not rely on absolute pointers between stack frames.

5.2.4 The Build-Flag Toggle

The dissertation's central comparison requires that the only variable between the two builds is the architectural placement of the migration mechanism. Every other aspect of

the system — the kernel code, the scheduler, the network stack, the test workload — must be identical. The mechanism that achieves this is a single Makefile flag, MIGRATION_MODE, that toggles a single #ifdef in the test harness. Figure 5.3 shows the two call paths that result.

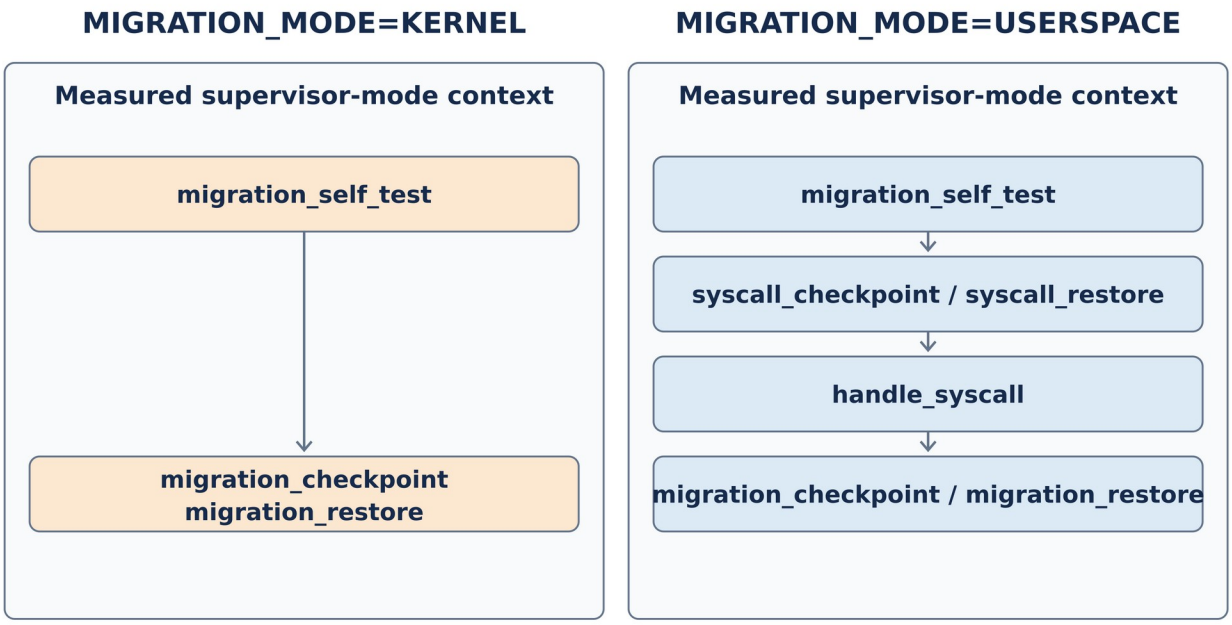


Figure 5.3: Two architectural placements compared by the dissertation, selectable by MIGRATION_MODE

In the kernel-resident mode (make MIGRATION_MODE=KERNEL), the test harness in apps/migration_test.c calls migration_checkpoint(...) directly as a kernel function call:

```
#if defined(MIGRATION_KERNEL_RESIDENT)
   long n = migration_checkpoint(pid, buf, sizeof(buf));
#else
   long n = syscall_checkpoint(pid, buf, sizeof(buf));
#endif
```

In the user-space mode (make MIGRATION_MODE=USERSPACE), the same line of source code becomes a call to syscall_checkpoint, which dispatches through handle_syscall and ultimately reaches the same migration_checkpoint implementation. The kernel binary in both builds contains both paths; only the path exercised by the benchmark differs.

The Makefile (Makefile, top-level) defines MIGRATION_MODE as defaulting to USERSPACE and exports the corresponding C preprocessor symbol (-DMIGRATION_KERNEL_RESIDENT=1 or -DMIGRATION_USERSPACE=1) via CFLAGS. The same Makefile also passes the QEMU runtime flag -global virtio-mmio.force-legacy=false required by the VirtIO-Net driver (see Section 5.2.6).

5.2.5 BENCH Instrumentation

The cycle-counter instrumentation is a single C macro defined in kernal/bench.h (69 lines total). The macro reads the RISC-V unprivileged cycle CSR (control and status register 0xC00) via the inline-assembly sequence csrr %0, cycle and emits a single ASCII line over the UART:

```
static inline unsigned long bench_rdcycle(void) {
    unsigned long c;
    __asm__ volatile("csrr %0, cycle" : "=r"(c));
    return c;
}


#define BENCH_TS(label) \
    do { \
        unsigned long _c = bench_rdcycle(); \
        printk("BENCH %s %ld\n", (label), _c); \
    } while (0)
```

An earlier implementation used the machine-mode mcycle CSR (0xB00). This was discovered to be unworkable because Hydra runs in supervisor mode rather than machine mode — OpenSBI's boot output explicitly reports Domain0 Next Mode : S-mode — and reading an M-mode CSR from S-mode is an illegal instruction that silently traps without producing any output. The fix was the one-character change from mcycle to cycle. The cycle CSR is accessible from S-mode and U-mode when mcounteren.cy is set, which OpenSBI does by default. The CSR returns the same counter value as mcycle would have returned, so no measurement fidelity is lost.

BENCH timestamps are placed at nine well-defined phase boundaries in the migration mechanism: ckpt_enter at entry to migration_checkpoint, ckpt_freeze immediately after

the source task is marked TASK_BLOCKED, ckpt_copied after the state has been copied into the user-provided buffer, send_start and send_done bracketing the (placeholder) transport phase in the local benchmark, recv_start and recv_done on the destination side of the cross-node benchmark, and rest_start and rest_done bracketing the restore call. Chapter 4 Section 4.2.2 derives five phase metrics from these nine timestamps.

A second printk format issue was discovered and fixed during the BENCH integration. The first version of the macro emitted BENCH %s %lu (unsigned long decimal), but Hydra's printk does not support %lu — it supports %ld and %lx. The mis-spec produced output that looked correct in some places but corrupted the variadic argument list in others, suppressing the BENCH lines entirely. The fix was to use %ld, which Hydra does support, and which prints the unsigned counter value correctly because the print routine ignores the signed/unsigned distinction at the C-level abstraction.

A third printk issue surfaced during the VirtIO-Net development: Hydra's printk had no support for zero-padded width modifiers, so %02x (used for MAC-address printing) was rendered literally. A 25-line addition to kernal/printk.c (a new print_num_padded helper plus a small parser extension) added %0Nx and %0Nd support for single-digit widths.

5.2.6 The VirtIO-Net 1.1 Modern MMIO Driver

The pre-existing kernal/net_driver.c was a stub: it probed the VirtIO MMIO magic register and read the device identifier, but its net_send and net_receive functions logged debug messages and returned without performing any virtqueue interaction. No actual Ethernet frames left QEMU through the original driver. For the cross-node migration work this was unworkable, and a complete replacement was written (now 322 lines).

The replacement targets the *modern* VirtIO MMIO transport (version 2), not the *legacy* (version 1) transport that QEMU's -device virtio-net-device exposes by default. The choice was forced by extended debugging of an initial legacy implementation that completed the QueuePFN-based setup correctly per the VirtIO 1.1 specification but did not produce any virtqueue_pop events in QEMU traces. The fix was to pass -global virtio-mmio.force-legacy=false to QEMU, which exposes the modern transport, and to

rewrite the driver around the modern register layout (separate QueueDescLow/High, QueueDriverLow/High, and QueueDeviceLow/High addresses instead of a single QueuePFN). The Makefile passes the flag for all run targets.

The driver scans the eight VirtIO MMIO slots starting at 0x10001000 looking for device ID 1 (network). On the QEMU `virt` machine the device appears at the high end of the slot range (0x10008000), not the low end as the default-base would suggest. The initialisation dance follows the modern spec: reset, acknowledge, set the driver bit, read the 64-bit device-feature mask in two 32-bit windows, write the accepted feature mask back, set the FEATURES_OK status bit, verify, set up each virtqueue, and finally set DRIVER_OK. The driver accepts VIRTIO_NET_F_MAC (bit 5) and VIRTIO_F_VERSION_1 (bit 32); all other offered features are declined.

Each virtqueue is configured with eight descriptors (the maximum the device exposes is 1 024, but eight is sufficient for the project's workload). The descriptor table, available ring, and used ring are pre-allocated as aligned `.bss` arrays so that no runtime allocation can fail. The send path writes the virtio-net header followed by the Ethernet frame into a slot-indexed buffer, populates the descriptor, increments the available index, and writes the queue number to QueueNotify. The receive path polls the used ring on each `net_receive()` call, copies completed packets into the software RX queue, and refills the descriptor for reuse.

The driver supports both single-instance and dual-instance QEMU configurations. In the single-instance case (`-netdev user`) the driver exchanges ARP frames with QEMU's slirp gateway, which is verified by the ARP probe in `apps/net_test.c`. In the dual-instance case (`-netdev socket,mcast=230.0.0.1:1234`) the driver exchanges frames between two Hydra instances connected via QEMU's UDP-multicast socket back-end, which is the substrate for the cross-node migration measurements in Chapter 4 Section 4.2.5.

5.2.7 Pre-Existing Issues Found and Fixed

Two pre-existing issues in the inherited Hydra codebase were discovered during the benchmark work and required fixes for the project's measurements to be reproducible.

`create_task` slot leak (fixed in `kernal/sched.c`). The upstream create_task function only ever appended new tasks at tasks[num_tasks] and never reused freed slots:

```
struct task *task = &tasks[num_tasks];
...
num_tasks++;
```

With MAX_TASKS = 16, after enough task creations the table fills and all subsequent calls fail with "task table full". For the project's 60-run benchmark, which creates a fresh victim task for each migration, this caused the benchmark to fail after approximately six iterations. The fix scans for a TASK_UNUSED slot first and falls back to growing num_tasks only if no unused slot exists.

`startup_counter` frozen (fixed in `kernal/main.c`). The main loop's startup_counter++ increment was placed inside a conditional that gated raytracer auto-start:

```
if (!raytracer_started && ++startup_counter > 1000) {
    raytracer_started = true;
    ...
}
```

Once raytracer_started became true, the counter stopped incrementing, and any subsequent code that gated on a later counter threshold (such as the 5 000-tick auto-trigger for the migration benchmark sweep) never fired. The fix moves the increment to the top of the main loop, outside any conditional, so the counter advances unconditionally.

Both fixes are documented as part of the dissertation's contribution and are credited explicitly in the project root CHANGES.md.

5.2.8 Test and Demonstration Applications

Four test and demonstration programs were written. Each is small (80–210 lines) and self-contained.

`apps/migration_test.c` (154 lines). Local round-trip test with payload-size sweep. Each invocation spawns a synthetic counter task, yields the scheduler to give the task a

chance to context-switch out, calls checkpoint, then immediately calls restore on the same buffer, then yields again to confirm the restored task is running. The four payload sizes (0, 4 KB, 16 KB, 64 KB) are cycled through across successive invocations. The build-flag conditional described in Section 5.2.4 selects whether the checkpoint and restore calls go through the syscall layer or directly into the kernel function.

`apps/fault_tolerance_demo.c` (142 lines). Eight-step demonstration described in Chapter 4 Section 4.2.6: spawn workload, yield, checkpoint, yield, inject fault, observe counter frozen, restore, observe counter resumed. The persistent in-memory buffer survives the simulated fault precisely because it lives in .bss rather than in the failed task's memory; this is a deliberate simplification of the real fault-tolerance model in which the checkpoint would be persisted off-node, but it is sufficient for the qualitative demonstration the chapter makes.

`apps/net_test.c` (approximately 100 lines). The VirtIO-Net smoke test. In single-node mode it sends one ARP probe to the slirp gateway (10.0.2.2) and polls for the reply. In two-node mode it sends twenty Hydra-tagged broadcast frames (ethertype 0x88B5, magic HYDR, with the local MAC's last byte as a node identifier) and counts how many of the peer's frames it receives. The test verifies that both directions of the network path work before any cross-node migration is attempted.

apps/cross_node_migration.c implements the cross-node path. The original benchmark protocol used a fifteen-byte fragment header, up to 1,471 checkpoint bytes per fragment, six frames for the minimum checkpoint and a role selected by the MAC address. It did not acknowledge or retry incomplete transfers. The current implementation includes later message-identification, acknowledgement, duplicate-handling and retry changes, so the original 8/15 result must be interpreted against its earlier protocol rather than the current source alone.

5.2.9 Benchmark Harness and Analysis Scripts

The benchmark harness is a shell-and-Python pipeline.

`bench/benchmark.sh` (85 lines). Runs the kernel under QEMU for a fixed wall-clock timeout, captures the serial output, strips carriage returns, and extracts BENCH lines into a per-mode log file under bench/results/.

`bench/two_node_test.sh` (approximately 55 lines). Launches two QEMU instances connected via UDP multicast, each with a distinct MAC address. Captures the serial output of both instances and the wire bytes via QEMU's filter-dump option into per-side pcap files. Reports the per-side BENCH-line counts and the recognised peer-frame count.

`bench/analyze.py` (207 lines). Reads the per-mode logs, groups BENCH lines into per-migration records (delimited by ckpt_enter), and computes the five derived metrics (freeze_time, serialize_t, transport_t, deserialize_t, total_latency) for each record. Outputs the mean ± 95% CI half-width and the p50/p95 percentiles per metric, plus the per-payload-size breakdown when BENCH_SIZE tag lines are present.

`bench/analyze_xmig.py` (approximately 80 lines). Same methodology applied to cross-node logs, computing the source fragmentation, destination reassembly, and destination restore costs separately.

`apps/micro_bench.c` (219 lines). Implements the null-system-call and IPC round-trip micro-benchmarks reported in Chapter 4 Section 4.2.9. Both benchmarks run as U-mode tasks and issue real ecall instructions, so that what is measured is a genuine privilege transition rather than the direct kernel function call used by the libhydra wrappers. Each benchmark returns its total cycle count through the task's U-mode exit code, which the supervisor-side driver reads and emits as BENCH_MICRO lines. The benchmarks are compiled out by default and are enabled with make MICRO_BENCH=1, so that the migration measurements reported in Section 4.2.1 to Section 4.2.6 are unaffected.

`bench/micro_bench.sh` (68 lines). Boots the kernel under QEMU once per repetition — the micro-benchmarks run during kernel initialisation rather than on demand, so repeated samples require repeated boots — and collects the BENCH_MICRO lines into

a single log. The script refuses to run unless the kernel was built with MICRO_BENCH=1, and warns if per-message IPC tracing was left enabled, because that tracing dominates the IPC measurement by roughly a factor of thirty (Section 4.2.9).

`bench/analyze_micro.py` (145 lines). Groups the BENCH_MICRO lines by boot, discards any boot that did not produce both measurements so that every ratio is computed from a matched pair, and reports the per-operation cycle costs with their 95% confidence interval half-widths together with the round-trip-to-null-system-call ratio that Chapter 4 Section 4.2.9 compares against Amoeba.

`bench/plot.py` (89 lines). Renders the latency-versus-payload-size figure (Chapter 4 Figure 4.2) using matplotlib.

`bench/plot_phases.py` (96 lines). Renders the per-phase stacked-bar figure (Chapter 4 Figure 4.3).

`bench/fig_variance.py`, `bench/fig_tcb_comparison.py`, `bench/fig_cross_node.py`, `bench/fig_fault_timeline.py`, `bench/fig_timeline.py`, `bench/fig_barriers.py`, `bench/fig_architecture.py`, `bench/fig_checkpoint_layout.py`, `bench/fig_two_paths.py`, `bench/fig_migration_sequence.py`, `bench/fig_amoeba_comparison.py`. Render the additional figures used in this report. Total matplotlib source approximately 1,000 lines.

All Python scripts are deterministic: re-running them against the same log files produces identical output.

5.3 Software Execution

This section describes how to build, run, and reproduce the measurements. Every command in this section has been verified against the current source tree.

5.3.1 Prerequisites

The build and run environment requires:

```
sudo apt install gcc-riscv64-unknown-elf qemu-system-misc tcpdump
python3 -m venv .venv
.venv/bin/pip install matplotlib
```

The cross-toolchain and QEMU are required for kernel build and execution. Matplotlib is required for figure generation.

5.3.2 Building the Kernel

The default build mode is user-space:

```
make MIGRATION_MODE=USERSPACE
```

The output is kernel.elf (approximately 80 kilobytes). The boot banner printed at runtime reports which mode was selected: Migration mode: USER-SPACE for the default, or Migration mode: KERNEL-RESIDENT after a make clean and rebuild with make MIGRATION_MODE=KERNEL.

5.3.3 Interactive Run

```
make run-net
```

This invokes QEMU with the VirtIO-Net device and the force-legacy=false flag required by the driver. Once the kernel boots and prints [main] Ready, the interactive UART accepts the following single-character commands:

Key	Action
d	Print distributed system status
m	Invoke the inherited (stubbed) kernel migration code path
f	Broadcast node-discovery requests
c	Run one checkpoint/restore self-test, emit BENCH lines
r	Run the fault-tolerance demonstration

The kernel also automatically runs 60 migration tests at boot (cycling through the four payload sizes) followed by one fault-tolerance demo, without any user input required.

5.3.4 Reproducing the Headline Measurements

To reproduce Table 4.1 (the user-space versus kernel-resident comparison):

```
# Build user-space mode and capture 60 runs
make MIGRATION_MODE=USERSPACE
timeout 200s qemu-system-riscv64 -machine virt -nographic \
   -kernel kernel.elf -global virtio-mmio.force-legacy=false \
   -netdev user,id=net0 -device virtio-net-device,netdev=net0 \
   < /dev/null 2>&1 \
   | tr -d '\r' | grep -aE '^(BENCH |BENCH_SIZE )' \
   > bench/results/USERSPACE.log


# Same for kernel-resident mode
make clean && make MIGRATION_MODE=KERNEL
# ... same QEMU command but redirecting to KERNEL.log ...


# Analyse
python3 bench/analyze.py bench/results/KERNEL.log \
                bench/results/USERSPACE.log
```

Each QEMU run takes approximately three minutes. The total reproduction takes about seven minutes including the analysis step. The output of `analyze.py` is the table reported in Chapter 4 Section 4.2.1.

5.3.5 Reproducing the Cross-Node Measurements

```
./bench/two_node_test.sh 30
python3 bench/analyze_xmig.py /tmp/hydra-node1.log /tmp/hydra-node2.log
```

The first command launches two QEMU instances networked via UDP multicast for thirty seconds. The second command computes the cross-node BENCH statistics reported in Chapter 4 Section 4.2.5.

5.3.6 Regenerating the Figures

```
.venv/bin/python bench/plot.py       bench/results/KERNEL.log
bench/results/USERSPACE.log
.venv/bin/python bench/plot_phases.py bench/results/KERNEL.log
bench/results/USERSPACE.log
.venv/bin/python bench/fig_variance.py
.venv/bin/python bench/fig_tcb_comparison.py
.venv/bin/python bench/fig_amoeba_comparison.py
# … additional fig_*.py scripts for the remaining figures …
```

The data-driven plots (plot.py, plot_phases.py, fig_variance.py) require the local-mode log files to be present. The architectural diagrams (fig_architecture.py, fig_checkpoint_layout.py, fig_two_paths.py, fig_timeline.py, fig_barriers.py) have no external data dependencies. The Amoeba comparison figures (fig_amoeba_comparison.py) carry their measured values inline, with provenance recorded in bench/amoeba/RESULTS.md, and likewise need no log files.

5.4 Chapter Summary

This chapter links the original dataset to its shared checkpoint/restore primitives, mode-selected invocation paths, BENCH timestamps and test harness. The historical checkpoint diagram and line counts describe the baseline, while the repository now contains later changes. The USERSPACE-labelled local benchmark still uses direct supervisor-mode wrappers; later isolated process-image and transport validation are separate work. This distinction is necessary to interpret the original timing data and to reproduce or extend it responsibly.

Chapter 6 — Discussion and Recommendations

This chapter interprets the original Hydra measurements in relation to the research motivation and questions. Section 6.2 separates observed costs from broader architectural and adoption arguments; Section 6.2.4 restates the experiment's limits. Section 6.3 proposes privilege-boundary, hardware, reliable-transport and representative-workload measurements needed before extending the conclusions.

6.1 Introduction

The dissertation was framed in Chapter 1 around one main research question and three supporting sub-questions:

- **Main**: where should process migration mechanism live on a modern RISC-V microkernel, inside the kernel or in user space?
- **RQ1**: is user-space migration feasible on a minimal RISC-V microkernel?
- **RQ2**: what is the cost of placing migration mechanism in user space rather than in the kernel?
- **RQ3**: has the 1993 Mach architectural verdict been overturned?

Chapter 4 reported the data necessary to answer each of these questions. This chapter draws the inferences from that data. The chapter is not a re-presentation of the measurements; it is the argument they support and the recommendations that follow from that argument.

The chapter is organised so that the discussion (Section 6.2) addresses the research questions in the order they were asked, and the recommendations (Section 6.3) address the limitations disclosed in Chapter 1 Section 1.7 and re-emphasised in Chapter 4 Section 4.2.10.

6.2 Discussion

6.2.1 Answering the Main Research Question

The local pooled total-latency ratio is 1.00 after rounding, based on sixty observations per mode across four equally represented sizes. Per-size mean ratios range from

approximately 0.96 to 1.03. The means and plotted uncertainty describe similar observations, but no equivalence margin, equivalence test or confidence interval for the ratio itself is reported. The result concerns supervisor-mode invocation paths under QEMU, not a hardware-isolated migration service.

The main question is therefore answered conditionally: in the measured Hydra baseline, the wrapper/dispatch path shows little additional pooled cost relative to direct calls to shared primitives. This is a useful reference point for the experiment motivated by Section 1.6. It does not establish that architectural placement is never a performance issue. TCB size and isolation remain important design arguments (Biggs, Lee and Heiser, 2018), but those arguments do not replace the missing privilege-boundary and hardware measurements.

The three central limits are the absence of a measured hardware-enforced privilege boundary, QEMU execution, and the bounded stack/context model without a migrated per-task heap. They are stated in Section 1.7 and determine the interpretation here. Modern IPC studies motivate a stronger follow-on experiment; they do not predict Hydra's unmeasured hardware cost (Gu et al., 2020; Chen et al., 2024).

6.2.2 The Milojičić Section 7.3 Adoption Barriers, Revisited

Milojičić et al. (1999, Section 7.3) identify application demand, infrastructure, simpler alternatives and sociological resistance as adoption barriers. The following re-evaluation concerns selected contemporary deployment contexts, using the literature available to this report. It is an analytical argument about motivation and context, separate from the synthetic counter measurements; it does not establish production readiness or measured applicability to GPU, confidential-computing or automotive workloads.

Figure 6.1 summarises the re-evaluation.

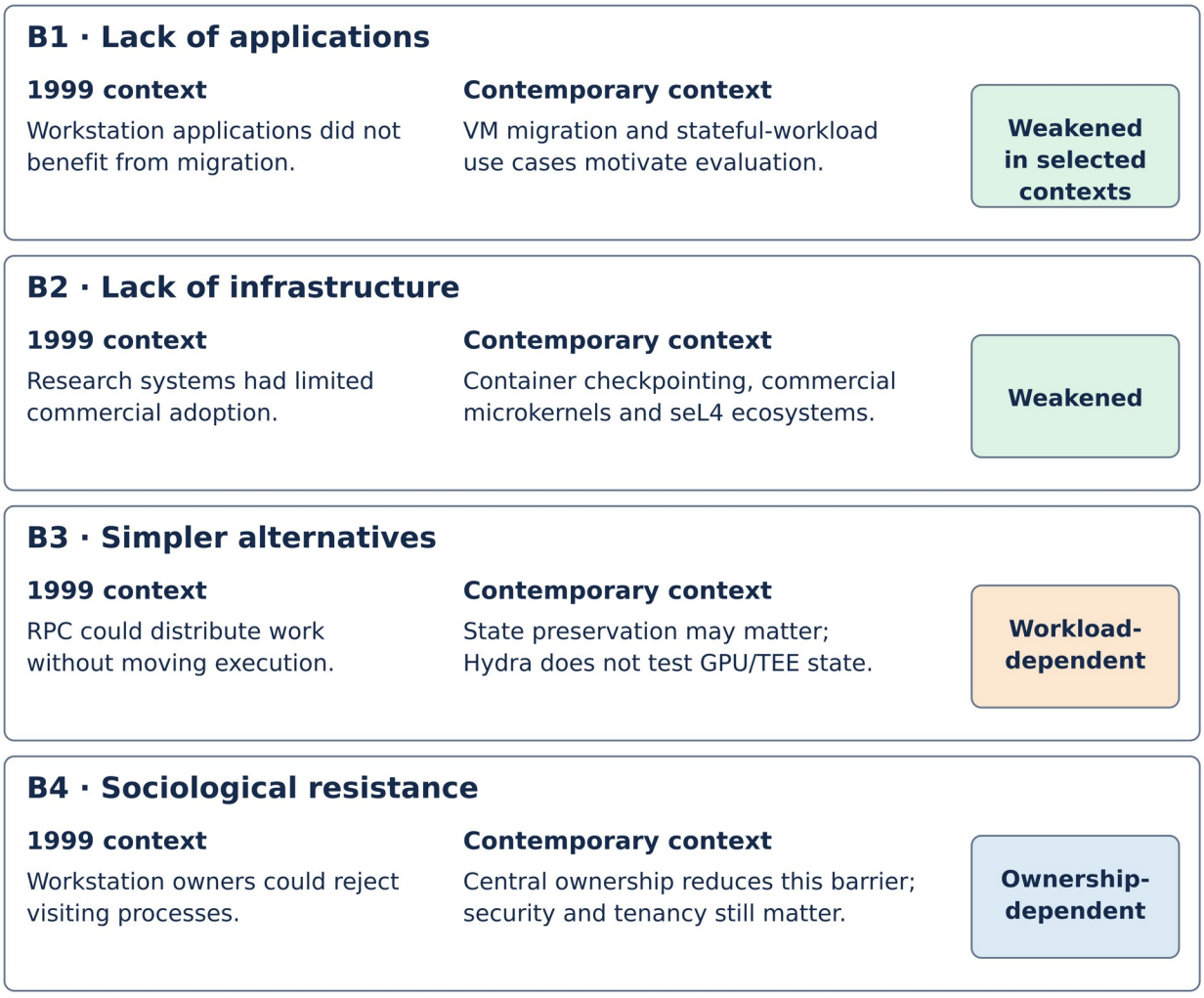


Figure 6.1: Adoption barriers from Milojičić et al. (1999), Section 7.3, reassessed by deployment context.

A historical timeline showing the relevant intervening events is also useful as context.

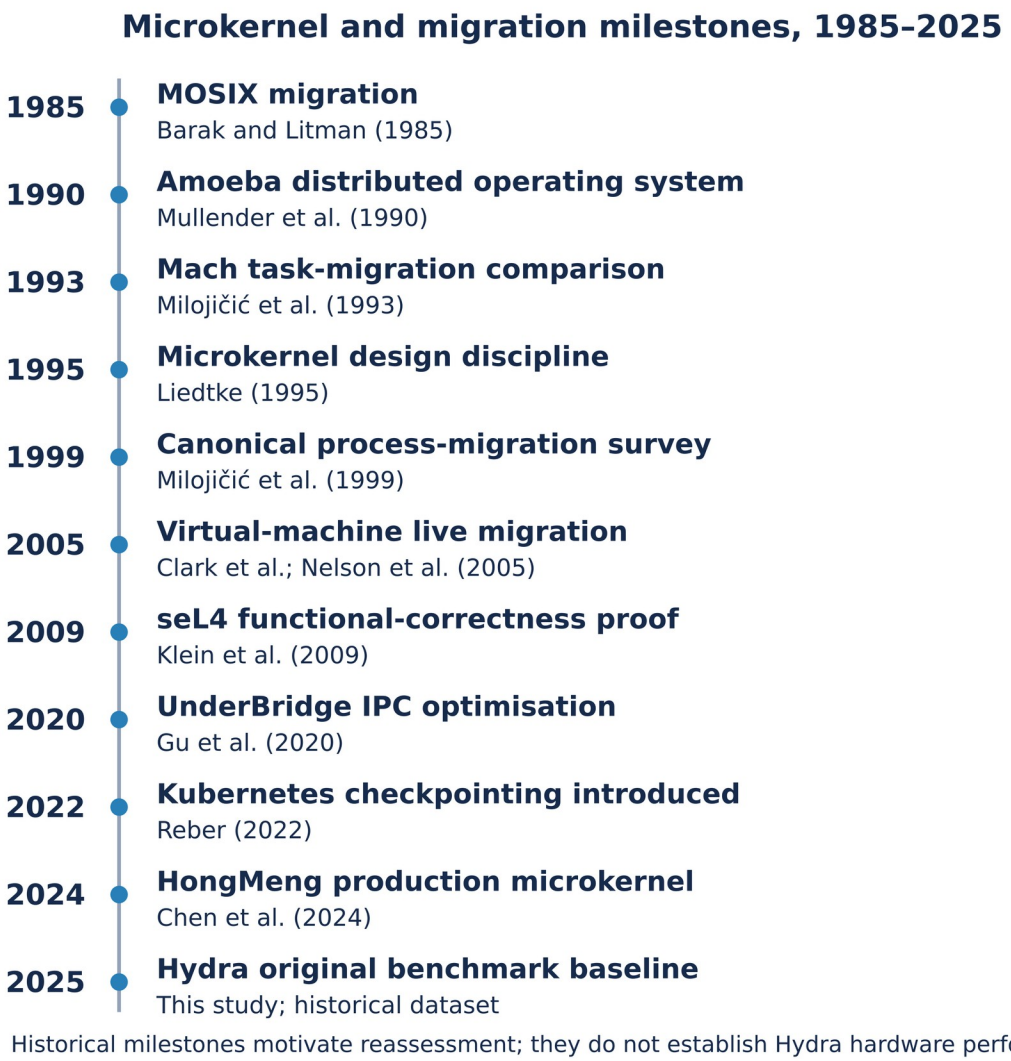


Figure 6.2: Microkernel and migration milestones, 1985–2025 (each event has a primary-source citation)

B1 — Lack of applications. Commercial virtual-machine migration establishes a practical state-preservation use case (Nelson, Lim and Hutchins, 2005; Clark et al., 2005). Kubernetes checkpointing integrations and automotive/edge use cases provide further motivation, with different maturity and state requirements (Reber, 2022; Avassa, 2026; Antmicro, 2020). Hydra does not measure those application classes. The evidence weakens the blanket lack-of-applications argument without demonstrating that its counter-task mechanism meets their needs.

Assessment: weakened in selected contexts. Demand for preserving execution state motivates fresh empirical study; it does not establish that every workload requires process migration.

B2 — Lack of infrastructure. Container runtimes, Kubernetes checkpoint support, commercial QNX systems and seL4-based initiatives provide more relevant infrastructure than the workstation research setting (Reber, 2022; BlackBerry, 2024; Google, 2022; Collins Aerospace, 2025). This does not mean that every platform supports live process migration or that migration is universally deployed. Reliable transfer, process-resource ownership and workload-specific integration still require evaluation.

Assessment: weakened, not eliminated. Available infrastructure makes migration experiments practical, while platform support and production maturity remain context-dependent.

B3 — Migration is not a requirement. In 1999 remote procedure call (Birrell and Nelson, 1984) and remote data access offered simpler ways to distribute work, removing the need to move live processes (Milojičić et al., 1999, p. 38). The dissertation argues that this barrier has not been overturned but has narrowed in scope. RPC and stateless web services remain the dominant pattern for most distributed workloads in 2025. However, three classes of contemporary state cannot be migrated by RPC: GPU-resident inference state of the kind held by large language model and vision workloads (NVIDIA, 2024); hardware-attested execution contexts in trusted execution environments such as Intel SGX, AMD SEV-SNP, and the RISC-V AP-TEE work (Gu et al., 2019; Confidential Computing Consortium, 2025); and long-running AI training jobs in which restart-from-scratch on hardware failure or pre-emption is not feasible.

Assessment: workload-dependent. RPC can be sufficient for stateless work, while preserving execution state may matter for other workloads. The Hydra benchmark does not implement or validate migration of GPU-resident state, hardware-attested contexts or large AI training jobs, so those remain motivations for future case studies.

B4 — Sociological factors. In 1999 the dominant deployment model was the workstation cluster, in which each workstation belonged to an individual user, and users were not inclined to allow remote processes to visit their machines (Krueger and Chawla, 1991; Milojičić et al., 1999, p. 38). The dissertation argues that this barrier has been rendered inapplicable rather than overturned. The workstation-cluster model has

been supplanted by three deployment models in which compute hardware is owned by a single operator: the cloud, the datacentre, and the embedded device (including the automotive zone controller). In all three models migration between hosts is the operator's decision and not the end user's.

Assessment: ownership-dependent. A single operator can remove the particular workstation-owner objection, but multi-tenant cloud, privacy, admission control and security policies can still restrict migration. The barrier is reduced in some centrally managed systems rather than universally inapplicable.

Summary. Application and infrastructure barriers are weaker in selected modern settings, simpler alternatives depend on workload, and sociological resistance depends on ownership and policy. This contextual reading supports Section 1.6's case for new measurements. It is not a numerical finding from the Hydra experiment or proof that commercial adoption barriers have disappeared.

6.2.3 Reassessing the 1993 Architectural Verdict

The 1993 Mach observation and Hydra's original rounded ratio of 1.00 concern materially different experiments (Milojičić et al., 1999, p. 29). The contrast supports reassessment, not a controlled refutation. Most importantly, Hydra's compared paths do not include the privilege separation whose overhead is central to the placement question.

The same three changes that motivate Section 1.6 also structure the next empirical steps:

- Reduced IPC cost motivates repeating the placement comparison with an actual protected boundary. UnderBridge and HongMeng show that mechanisms and deployment choices matter, but their results do not supply the missing Hydra measurement (Gu et al., 2020; Chen et al., 2024).
- Commercial state-preservation demand motivates tests with realistic resources and reliability targets. A low counter-task cost is not a substitute for measuring application state, delivery failures and recovery on the intended workload (Nelson, Lim and Hutchins, 2005; Reber, 2022).

- RISC-V adoption motivates an open baseline and physical RISC-V replication (Waterman and Asanović, 2019; RISC-V International, 2025). It does not justify transferring QEMU cycle counts or results from other ISAs directly to silicon.

The architectural conclusion is correspondingly limited: the historical Mach result should not be treated as a universal placement rule, and Hydra supplies initial evidence for a renewed comparison. A user-space recommendation on performance grounds remains conditional on real privilege-boundary, hardware and representative-workload measurements. Smaller kernel mechanisms may be attractive on isolation and reviewability grounds, but code size alone does not establish security or performance equivalence.

6.2.4 Limitations of the Present Study

These limitations apply to the original benchmark, even though the repository has subsequently evolved. They repeat the limits stated in Section 1.7 so that the conclusion cannot be read without its experimental scope.

- **Privilege boundary.** The USERSPACE-labelled comparison uses supervisor-mode C wrappers and handle_syscall. It does not include an isolated server's privilege transitions, address-space switches or protection checks. The observed ratio is neither a measured isolated-server cost nor a calibrated bound for another kernel. Section 6.3.1 defines the required follow-on measurement.
- **Emulation.** QEMU cycle observations are conditional on the emulator, guest and host configuration. Within-setup ratios and trends may guide further work, but they are not guaranteed on physical RISC-V hardware. Absolute counts must not be presented as hardware latency, deadline compliance or deployable throughput.
- **Bounded process state.** The historical counter model has a fixed 8 KB stack, saved context and synthetic extra bytes, without migrated private per-task heaps or external resource state. Copying a larger synthetic buffer does not demonstrate correct restoration of heap pointers, files, sockets or device ownership. Later isolated process-image support is separate engineering evidence, not part of this timing sample.

- **Cross-node reliability and selection.** Eight of fifteen original attempts complete under best-effort multicast without ACK/retry; seven time out. The 46.7% attempt failure proportion is not a per-fragment loss estimate. Destination phase means describe completions only, and source/destination samples have different counts. Section 6.3.2 requires measurement of all attempts and retry costs under the improved protocol.
- **Generalisation and provenance.** Only the selected RISC-V/QEMU configuration and counter workload are measured. Fifteen repetitions per size/mode cell do not prove equivalence or independence. Historical source/binary identity, full raw UART logs and host/emulator settings are needed for exact replication; current code and later validation counts must not be silently combined with the saved original summaries.
- **Pre-existing kernel quality.** The dissertation discovered two material bugs in the inherited Hydra codebase (the `create_task` slot leak and the `startup_counter` freeze) and three `printk` issues (the `mcycle` versus `cycle` CSR issue, the unsupported `%lu` format specifier, and the missing `%0Nx` width parser). All five were fixed, the fixes are credited to the dissertation in `CHANGES.md`, and they are documented in Chapter 5 Section 5.2.5 and Section 5.2.7. The fact that the benchmark work surfaced these issues suggests that Hydra is suitable as a teaching kernel but would require additional hardening before it could be considered a research baseline.

6.3 Recommendations

Four recommendations follow from the discussion. They are presented in approximately decreasing order of dissertation relevance.

6.3.1 Real Privilege-Boundary Measurement on seL4 RISC-V

Repeat the matched placement experiment with a hardware-enforced user/kernel boundary, either on the later isolated Hydra path or on seL4's QEMU RISC-V platform (seL4 Foundation, n.d.). Measure actual transitions, IPC/address-space effects and first-resumed-instruction latency with reduced logging overhead. Use the same

workload and sizes initially, define a practically meaningful equivalence margin in advance, collect independent sessions and report uncertainty for differences or ratios. Then repeat on physical RISC-V hardware with documented counter calibration, board, clock and toolchain settings. The outcome must be measured; it cannot be predicted from UnderBridge's Intel-PKU result.

6.3.2 Cross-Node Migration with a Reliable Transport

The original 7/15 attempt failures motivate an acknowledged transport with migration IDs, duplicate detection, retransmission and bounded timeouts. The repository's subsequent ACK/retry implementation and recorded 15/15 supervisor validation show engineering progress, but those later runs are not the original study's samples or proof of production reliability. Re-run a matched experiment against the original baseline, include incomplete/failed attempts, inject loss and delay, and report completion rate, retry count, total time and tail latency. This directly addresses Section 4.2.5 without promising a near-100% success rate before evaluation.

6.3.3 Software-Defined-Vehicle Workload Case Study

A software-defined-vehicle case study would connect the commercial motivation in Section 1.6 to a representative workload. Migrate a sensor-processing or controller task between nodes, then verify state, external-resource ownership, failure recovery and deadlines on the intended hardware. The present counter benchmark does not establish real-time suitability, certification or safe deployment (BlackBerry, 2024; Avassa, 2026).

6.3.4 Edge-AI Inference Workload Migration

An edge-inference case study would test whether state-preserving migration benefits a representative RISC-V workload (Antmicro, 2020; seL4 Foundation, 2025). It should define which model weights, mutable state and resources move, distinguish restartable/stateless work from execution that must be preserved, and verify restoration before measuring cost. This would extend the current 0–64 KB synthetic-extra sweep rather than treating it as evidence of real heap or accelerator-state migration.

6.4 Chapter Summary

The study provides an initial cycle-level baseline, not a universal architectural verdict. Its pooled total-latency ratio rounds to 1.00 for two supervisor-mode call paths, under QEMU, with a bounded counter process and synthetic extra payload. Reduced IPC cost, commercial state-preservation demand and RISC-V adoption justify the experiment and its extension; they do not establish equivalence across a real protection boundary. The original cross-node result of eight completions in fifteen attempts exposes the limits of best-effort UDP without retransmission and directly motivates the renewed transport evaluation in Section 6.3.2. Physical-hardware, privilege-separated and realistic-process measurements are required before generalising the absolute counts, ratios or application suitability. Later repository improvements are acknowledged as separate work rather than substituted into the original dataset.

References

References use author–date Harvard formatting and are ordered alphabetically. In-text author/year keys have been cross-checked against these entries. Undated project documentation is cited as n.d.; access dates identify the consulted web versions. The 1999 technical report and 2000 journal version of the Milojičić survey remain separate sources.

Accetta, M., Baron, R., Bolosky, W., Golub, D., Rashid, R., Tevanian, A. and Young, M. (1986) 'Mach: A New Kernel Foundation for UNIX Development', *Proceedings of the USENIX Summer Conference*, pp. 93–112.

Antmicro (2020) *Building RISC-V Systems on the Secure seL4 Microkernel in Renode*. Available at: https://antmicro.com/blog/2020/09/sel4-on-riscv-in-renode/ (Accessed: 14 May 2026).

Avassa (2026) *Software-Defined Vehicle Platform: 2026 Development Guide*. Available at: https://avassa.io/articles/key-considerations-for-a-software-defined-vehicle-development-platform/ (Accessed: 14 May 2026).

Barak, A. and Litman, A. (1985) 'MOS: A Multicomputer Distributed Operating System', *Software — Practice and Experience*, 15(8), pp. 725–737.

Barbalace, A., Karaoui, M.L., Wang, W., Xing, T., Olivier, P. and Ravindran, B. (2020) 'Edge Computing – the Case for Heterogeneous-ISA Container Migration', *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE '20)*. doi: 10.1145/3381052.3381321.

Bellard, F. (2005) 'QEMU, a Fast and Portable Dynamic Translator', *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, pp. 41–46.

Biggs, S., Lee, D. and Heiser, G. (2018) 'The Jury Is In: Monolithic OS Design Is Flawed', *Proceedings of the Asia-Pacific Workshop on Systems (APSys '18), 7 pp*. doi: 10.1145/3265723.3265733.

Birrell, A.D. and Nelson, B.J. (1984) 'Implementing Remote Procedure Calls', *ACM Transactions on Computer Systems*, 2(1), pp. 39–59.

BlackBerry (2024) *Software-Defined Vehicles: The Ultimate Guide*. Available at: https://blackberry.qnx.com/en/ultimate-guides/software-defined-vehicle (Accessed: 14 May 2026).

Bosse, S. (2002) FSD-Amoeba 2002B: A Free Software Distribution of the Amoeba Distributed Operating System. Available at: https://sourceforge.net/projects/fsd-amoeba/ (Accessed: 20 September 2026).

Boyd-Wickizer, S., Clements, A.T., Mao, Y., Pesterev, A., Kaashoek, M.F., Morris, R. and Zeldovich, N. (2010) 'An Analysis of Linux Scalability to Many Cores', *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*, pp. 1–16.

Chen, H., Miao, X., Jia, N., Wang, N., Li, Y., Liu, N., Liu, Y., Wang, F., Huang, Q., Li, K., Yang, H., Wang, H., Yin, J., Peng, Y. and Xu, F. (2024) 'Microkernel Goes General: Performance and Compatibility in the HongMeng Production Microkernel', *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), pp. 465–485*. Available at: https://www.usenix.org/conference/osdi24/presentation/chen-haibo (Accessed: 13 October 2026).

Cheriton, D.R. (1988) 'The V Distributed System', *Communications of the ACM*, 31(3), pp. 314–333.

Clark, C., Fraser, K., Hand, S., Hansen, J.G., Jul, E., Limpach, C., Pratt, I. and Warfield, A. (2005) 'Live Migration of Virtual Machines', *Proceedings of the 2nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '05)*, pp. 273–286.

Collins Aerospace (2025) *INSPECTA: Integrated, Secure, Provable, Embedded Cyber Trusted Architecture*. seL4 Summit 2025 Abstracts. Available at: https://sel4.systems/Summit/2025/abstracts2025.html (Accessed: 14 May 2026).

Confidential Computing Consortium (2025) *Confidential Computing: The Next Frontier in Data Security*. White Paper. Available at: https://confidentialcomputing.io/ (Accessed: 14 May 2026).

Connected Automated Driving (2025) *The Technology Roadmap 2025 for the Software-Defined Vehicle*. Available at: https://www.connectedautomateddriving.eu/blog/the-technology-roadmap-2025-for-the-software-defined-vehicle-sdvsw-platform-is-available/ (Accessed: 14 May 2026).

CRIU project (n.d.) Checkpoint/Restore. Available at: https://criu.org/Checkpoint/Restore (Accessed: 13 October 2026).

Douglis, F. and Ousterhout, J. (1991) 'Transparent Process Migration: Design Alternatives and the Sprite Implementation', *Software — Practice and Experience*, 21(8), pp. 757–785.

Google (2022) *KataOS: An Open-Source Operating System for Ambient ML Applications*. Available at: https://opensource.googleblog.com/2022/10/announcing-kataos-and-sparrow.html (Accessed: 14 May 2026).

Gu, J., Hua, Z., Xia, Y., Chen, H. and Zang, B. (2019) 'MicroTEE: Designing TEE OS Based on the Microkernel Architecture', *Proceedings of the 18th IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom '19)*, pp. 26–33.

Gu, J., Wu, X., Li, W., Liu, N., Mi, Z., Xia, Y. and Chen, H. (2020) 'Harmonizing Performance and Isolation in Microkernels with Efficient Intra-kernel Isolation and Communication', *Proceedings of the 2020 USENIX Annual Technical Conference*, pp. 401–417.

Heiser, G., Elphinstone, K., Kuz, I., Klein, G. and Petters, S.M. (2007) 'Towards Trustworthy Computing Systems: Taking Microkernels to the Next Level', *ACM SIGOPS Operating Systems Review, 41(4), pp. 3–11*. doi: 10.1145/1278901.1278904.

Heiser, G., Klein, G. and Murray, T. (2019) 'Can We Prove Time Protection?', *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '19)*, pp. 23–29.

Heiser, G. (2020) 'The seL4 Microkernel: An Introduction', *seL4 Foundation White Paper*. Available at: https://sel4.systems/About/seL4-whitepaper.pdf (Accessed: 14 May 2026).

Jain, R. (1991) *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. New York: John Wiley & Sons.

Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H. and Winwood, S. (2009) 'seL4: Formal Verification of an OS Kernel', *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP '09)*, pp. 207–220.

Krueger, P. and Chawla, R. (1991) 'The Stealth Distributed Scheduler', *Proceedings of the 11th International Conference on Distributed Computing Systems (ICDCS '91)*, pp. 336–343.

Lampson, B., Abadi, M., Burrows, M. and Wobber, E. (1992) 'Authentication in Distributed Systems: Theory and Practice', *ACM Transactions on Computer Systems*, 10(4), pp. 265–310.

Liedtke, J. (1995) 'On µ-Kernel Construction', *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP '95)*, pp. 237–250.

Milojičić, D., Zint, W., Dangel, A. and Giese, P. (1993) 'Task Migration on Top of the Mach Microkernel', *Proceedings of the Third USENIX Mach Symposium*, pp. 273 –290.

Milojičić, D., Douglis, F., Paindaveine, Y., Wheeler, R. and Zhou, S. (1999) *Process Migration*. HP Laboratories Technical Report HPL-1999-21.

Milojičić, D., Douglis, F., Paindaveine, Y., Wheeler, R. and Zhou, S. (2000) 'Process Migration', *ACM Computing Surveys*, 32(3), pp. 241–299.

Mullender, S.J., van Rossum, G., Tanenbaum, A.S., van Renesse, R. and van Staveren, H. (1990) 'Amoeba — A Distributed Operating System for the 1990s', *IEEE Computer*, 23(5), pp. 44–53.

Murray, T., Matichuk, D., Brassil, M., Gammie, P., Bourke, T., Seefried, S., Lewis, C., Gao, X. and Klein, G. (2013) 'seL4: From General Purpose to a Proof of Information Flow Enforcement', *Proceedings of the 34th IEEE Symposium on Security and Privacy*, pp. 415–429.

Nelson, M., Lim, B.-H. and Hutchins, G. (2005) 'Fast Transparent Migration for Virtual Machines', *Proceedings of the USENIX Annual Technical Conference*, pp. 391–394.

NVIDIA (2024) *Confidential Computing on NVIDIA Hopper GPUs*. Technical Brief. Available at: https://www.nvidia.com/en-us/data-center/solutions/confidential-computing/ (Accessed: 14 May 2026).

Powell, M.L. and Miller, B.P. (1983) 'Process Migration in DEMOS/MP', *Proceedings of the 9th ACM Symposium on Operating Systems Principles (SOSP '83)*, pp. 110–119.

Rashid, R. and Robertson, G. (1981) 'Accent: A Communication-Oriented Network Operating System Kernel', *Proceedings of the 8th ACM Symposium on Operating Systems Principles (SOSP '81)*, pp. 64–75.

Reber, A. (2022) 'Forensic container checkpointing in Kubernetes', *Kubernetes Blog, 5 December*. Available at: https://kubernetes.io/blog/2022/12/05/forensic-container-checkpointing-alpha/ (Accessed: 13 October 2026).

Research and Markets (2025) *Microkernel Real-Time Operating System Market — Global Forecast Report 2025–2030*. Available at: https://www.researchandmarkets.com/ (Accessed: 14 May 2026).

RISC-V International (2025) *RISC-V Annual Report 2025*. Available at: https://riscv.org/wp-content/uploads/2026/01/RISC-V-Annual-Report-2025.pdf (Accessed: 14 May 2026).

Roland Berger (2024) *Explaining Software-Defined Vehicles*. Available at: https://www.rolandberger.com/en/Insights/Publications/Explaining-software-defined-vehicles.html (Accessed: 14 May 2026).

seL4 Foundation (n.d.) QEMU RISC-V virtual platform. Available at: https://docs.sel4.systems/Hardware/qemu-riscv-virt.html (Accessed: 13 October 2026).

seL4 Foundation (2025) *seL4 Summit 2025 Abstracts*. Available at: https://sel4.systems/Summit/2025/abstracts2025.html (Accessed: 14 May 2026).

Tanenbaum, A.S., van Renesse, R., van Staveren, H., Sharp, G.J., Mullender, S.J., Jansen, J. and van Rossum, G. (1990) 'Experiences with the Amoeba Distributed Operating System', Communications of the ACM, 33(12), pp. 46–63.

Tanenbaum, A.S. and Bos, H. (2014) *Modern Operating Systems*. 4th edn. Upper Saddle River, NJ: Pearson.

Walker, B., Popek, G., English, R., Kline, C. and Thiel, G. (1983) 'The LOCUS Distributed Operating System', *Proceedings of the 9th ACM Symposium on Operating Systems Principles (SOSP '83)*, pp. 49–70.

Waterman, A. and Asanović, K. (2019) *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Version 20191213. Available at: https://riscv.org/specifications/ (Accessed: 14 May 2026).

Watson, R.N.M., Neumann, P.G., Woodruff, J., Roe, M., Almatary, H., Anderson, J., Baldwin, J., Chisnall, D., Clarke, J., Davis, B., Eisen, L., Filardo, N.W., Grisenthwaite, R., Joannou, A., Laurie, B., Markettos, A.T., Moore, S.W., Murdoch, S.J., Nienhuis, K., Norton, R., Richardson, A., Rugg, P., Sewell, P., Son, S. and Xia, H. (2020) *Capability Hardware Enhanced RISC Instructions: CHERI Instruction-Set Architecture (Version 8)*. Technical Report UCAM-CL-TR-951, University of Cambridge.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B. and Wesslén, A. (2012) *Experimentation in Software Engineering*. Berlin: Springer.

Zhou, S., Zheng, X., Wang, J. and Delisle, P. (1994) 'Utopia: A Load Sharing Facility for Large, Heterogeneous Distributed Computer Systems', *Software — Practice and Experience*, 23(12), pp. 1305–1336.